

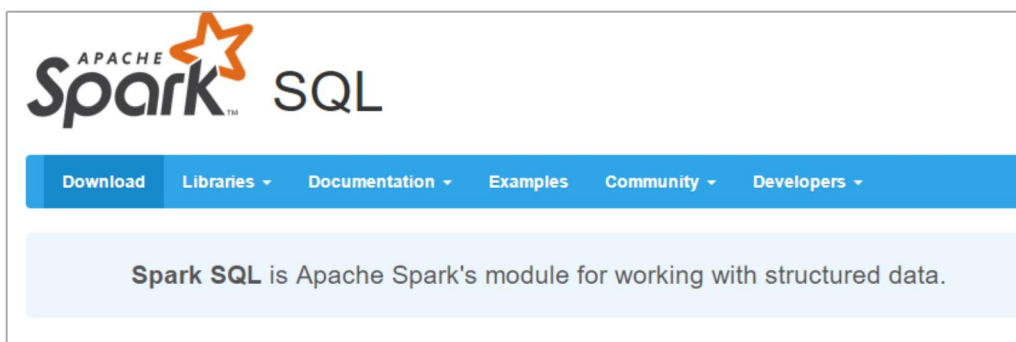
【多易教育】SparkSql 教程



JUST DO IT
多易教育

第 1 章 Spark SQL 概述

1.1 Spark SQL 定义



Spark SQL 是基于 spark core 提供的一个用来处理结构化数据的模块（库）

它提供了一个编程抽象叫做 **DataFrame/Dataset**，它可以理解为一个基于 RDD 数据模型的更高级数据模型，带有结构化元信息（schema）

DataFrame 其实就是 **Dataset[Row]**

```
* id,name,age,city,score
* 1,张飞,21,北京,80.0
* 2,关羽,23,北京,82.0
* 3,赵云,20,上海,88.6
* 4,刘备,26,上海,83.0
* 5,曹操,30,深圳,90.0
*/

val dataframe = spark.read.options(Map("header" -> "true", "delimiter" -> ";"))
    .csv(path = "data_ware/demodata/stu.txt")
dataframe.printSchema() // 打印schema信息
dataframe.show() // 打印“表”数据

JiaoAn > main(args: Array[String])
JiaoAn x
D:\devapp\Java\jdk1.8.0_102\bin\java.exe ...
Using Spark's default log4j profile: org/apache/
root
|-- id: integer (nullable = true)
|-- name: string (nullable = true)
|-- age: integer (nullable = true)
|-- city: string (nullable = true)
|-- score: double (nullable = true)

+---+---+---+---+---+
| id|name|age|city|score|
+---+---+---+---+---+
| 1| 张飞| 21| 北京| 80.0|
| 2| 关羽| 23| 北京| 82.0|
| 3| 赵云| 20| 上海| 88.6|
| 4| 刘备| 26| 上海| 83.0|
| 5| 曹操| 30| 深圳| 90.0|
+---+---+---+---+---+
```

Spark SQL 可以将针对 DataFrame/Dataset 的各类 SQL 运算，翻译成 RDD 的各类算子执行计划，从而大大简化数据运算编程（请联想 Hive）

```
dataFrame.createTempView( viewName = "stu")
val res = spark.sql(
  """
    |select city,avg(score) as avg_sc
    |from stu
    |group by city
    """.stripMargin)

res.show()
```

JiaoAn > main(args: Array[String])

JiaoAn x

D:\devapp\Java\jdk1.8.0_102\bin\java.exe ...

Using Spark's default log4j profile: org/apache/...

```
+-----+-----+
|city|avg_sc|
+-----+-----+
| 深圳|  90.0|
| 上海|  85.8|
| 北京|  81.0|
+-----+-----+
```

```
res.explain(extended = true)
```

JiaoAn > main(args: Array[String])

JiaoAn x

D:\devapp\Java\jdk1.8.0_102\bin\java.exe ...

Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties

```
== Parsed Logical Plan ==
'Aggregate ['city], ['city, 'avg('score) AS avg_sc#24]
+- 'UnresolvedRelation `stu`

== Analyzed Logical Plan ==
city: string, avg_sc: double
Aggregate [city#15], [city#15, avg(score#16) AS avg_sc#24]
+- SubqueryAlias stu
   +- Relation[id#12,name#13,age#14,city#15,score#16] csv

== Optimized Logical Plan ==
Aggregate [city#15], [city#15, avg(score#16) AS avg_sc#24]
+- Project [city#15, score#16]
   +- Relation[id#12,name#13,age#14,city#15,score#16] csv

== Physical Plan ==
*HashAggregate(keys=[city#15], functions=[avg(score#16)], output=[city#15, avg_sc#24])
+- Exchange hashpartitioning(city#15, 200)
   +- *HashAggregate(keys=[city#15], functions=[partial_avg(score#16)], output=[city#15, sum#33, c
      +- *FileScan csv [city#15,score#16] Batched: false, Format: CSV, Location: InMemoryFileIndex
```

1.2 Spark SQL 的特性

1. 易整合

Integrated

Seamlessly mix SQL queries with Spark programs.

Spark SQL lets you query structured data inside Spark programs, using either SQL or a familiar [DataFrame API](#). Usable in Java, Scala, Python and R.

```
results = spark.sql(
    "SELECT * FROM people")
names = results.map(lambda p: p.name)
```

Apply functions to results of SQL queries.

Spark SQL 使得在 spark 编程中可以如丝般顺滑地混搭 SQL 和算子 api 编程（想想都激动不是！）

2. 统一的数据访问方式

Uniform Data Access

Connect to any data source the same way.

DataFrames and SQL provide a common way to access a variety of data sources, including Hive, Avro, Parquet, ORC, JSON, and JDBC. You can even join data across these sources.

```
spark.read.json("s3n://...")
    .registerTempTable("json")
results = spark.sql(
    """SELECT *
    FROM people
    JOIN json ...""")
```

Query and join different data sources.

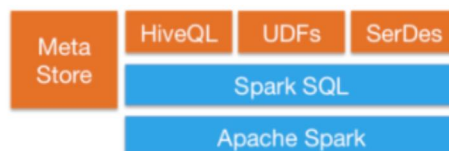
Spark SQL 为各类不同数据源提供统一的访问方式，可以跨各类数据源进行愉快的 join；所支持的数据源包括但不限于：Hive / Avro / CSV / Parquet / ORC / JSON / JDBC 等；（简直太太好了！）

3. 兼容 Hive

Hive Integration

Run SQL or HiveQL queries on existing warehouses.

Spark SQL supports the HiveQL syntax as well as Hive SerDes and UDFs, allowing you to access existing Hive warehouses.



Spark SQL can use existing Hive metastores, SerDes, and UDFs.

Spark SQL 支持 HiveQL 语法及 Hive 的 SerDes、UDFs，并允许你访问已经存在的 Hive 数仓数据；（难以置信的贴心！）

4. 标准的数据连接

Standard Connectivity

Connect through JDBC or ODBC.

A server mode provides industry standard JDBC and ODBC connectivity for business intelligence tools.

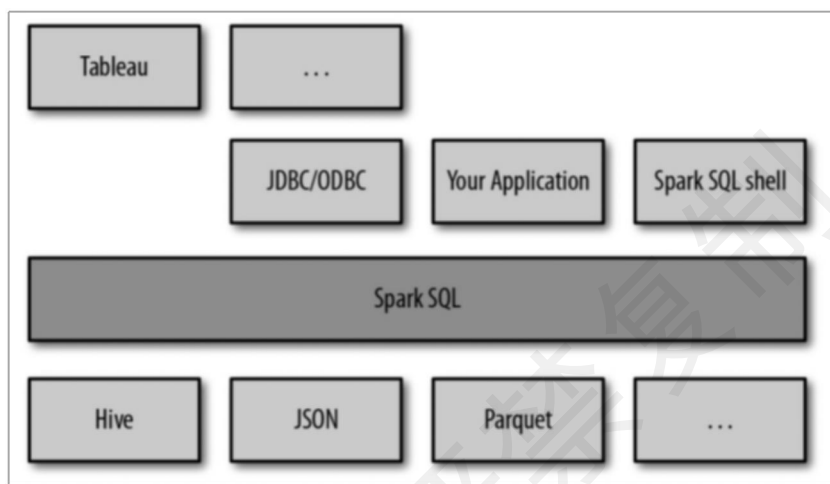
Use your existing BI tools to query big data.

BI Tools ...

JDBC / ODBC

Spark SQL

Spark SQL 的 server 模式可为各类 BI 工具提供行业标准的 JDBC/ODBC 连接，从而可以为支持标准 JDBC/ODBC 连接的工具提供无缝对接；（开发封装平台很有用哦！）



SparkSQL 可以看做一个转换层，向下对接各种不同的结构化数据源，向上提供不同的数据访问方式。

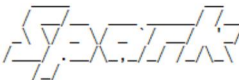
第 2 章 SparkSQL 快速体验

2.1 命令行使用示例

示例需求：查询大于 30 岁的用户

1. 启动 Spark shell

```
[root@doit01 spark-2.4.0]# bin/spark-shell
2019-10-07 11:43:35 WARN NativeCodeLoader:62 - unable to load native-hadoop library
va classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(new
spark context web UI available at http://doit01:4040
Spark context available as 'sc' (master = local[*], app id = local-1570419822429).
Spark session available as 'spark'.
Welcome to

 version 2.4.0

Using Scala version 2.11.12 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_191)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

2. 创建如下 JSON 文件，注意 JSON 的格式：

```
{ "name": "Michael" }
{ "name": "Andy", "age": 30 }
{ "name": "Justin", "age": 19 }
{ "name": "brown", "age": 39 }
{ "name": "jassie", "age": 34 }
```

```
{ "name": "Michael" }
{ "name": "Andy", "age": 30 }
{ "name": "Justin", "age": 19 }
{ "name": "brown", "age": 39 }
{ "name": "jassie", "age": 34 }
```

3. 愉快地开始使用：

```
scala> val df = spark.read.json("/root/people.json")
df: org.apache.spark.sql.DataFrame = [age: bigint, name: string]

scala> df.printSchema
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)

scala> df.show
+----+-----+
| age | name |
+----+-----+
| null | Michael |
| 30 | Andy |
| 19 | Justin |
+----+-----+

scala> df.filter('age > 21).show
+----+-----+
| age | name |
+----+-----+
| 30 | Andy |
+----+-----+

scala> df.createTempview("p")

scala> spark.sql("SELECT * FROM p WHERE age > 21").show
2019-10-07 11:48:12 WARN objectStore:568 - Failed to get database gl
+----+-----+
| age | name |
+----+-----+
| 30 | Andy |
+----+-----+
```

2.2 新的编程入口 SparkSession

在老的版本中，SparkSQL 提供两种 SQL 查询起始点，一个叫 SQLContext，用于 Spark 自己提供的 SQL 查询，一个叫 HiveContext，用于连接 Hive 的查询，SparkSession 是 Spark 最新的 SQL 查询起始点，实质上是 SQLContext 和 SparkContext 的组合，所以在 SQLContext 和 HiveContext 上可用的 API 在 SparkSession 上同样是可以使用的。SparkSession 内部封装了 sparkContext，所以计算实际上是由 sparkContext 完成的。

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession
  .builder()
  .appName("Spark SQL basic example")
  .config("spark.some.config.option", "some-value")
  .getOrCreate()

// 提供隐式转换支持，如 RDDs to DataFrames
import spark.implicits._
```

SparkSession.builder 用于创建一个 SparkSession。

import spark.implicits._ 的引入是用于将 DataFrames 隐式转换成 RDD，使 df 能够使用 RDD 中的方法。

如果需要 Hive 支持，则需要以下创建语句：

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession
    .builder( )
    .appName("Spark SQL basic example")
    .config("spark.some.config.option", "some-value")
    .enableHiveSupport( )
    .getOrCreate( )

// For implicit conversions like converting RDDs to DataFrames
import spark.implicits._
```

2.3 IDEA 开发 SparkSQL 程序

IDEA 中 SparkSQL 程序的开发方式和 SparkCore 类似。

添加 Maven 依赖：

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.11</artifactId>
  <version>2.2.0</version>
</dependency>
```

程序如下：

```
/**
 * @date: 2019/10/8
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description: 最基本的 sparksql 入门体验示例程序
 *              需求： 从 people.json 中进行数据分析，比如查询年龄>30 岁人的信息
 *
 */
object Demo1_HellWorld {

    // 屏蔽掉 WARN 级别以下的日志 (INFO,DEBUG)
    Logger.getLogger("org").setLevel(Level.WARN)

    def main(args: Array[String]): Unit = {
```

```
// 创建一个 sparksql 的编程入口(包含 sparkcontext, 也包含 sqlcontext)
val spark: SparkSession = SparkSession
    .builder()
    .appName(this.getClass.getSimpleName)
    .master("local[*]")
    .getOrCreate()

/*val sc: SparkContext = spark.sparkContext
val sqlsc: SQLContext = spark.sqlContext*/

// 加载 json 数据文件为 dataframe
val df: DataFrame = spark.read.json("data/people.dat")

// 打印 df 中的 schema 元信息
df.printSchema()

// 打印 df 中的数据
df.show(50, false)

// 在 df 上, 用调 api 方法的形式实现 sql
df.where("age > 30").show()

// 将 df 注册成一个“表”(视图), 然后写原汁原味的 sql
df.createTempView("people")
spark.sql("select * from people where age > 30 order by age desc").show()

spark.close()
}
}
```

第 3 章 DataFrame 编程详解

3.1 加载数据源为 DF

在 Spark SQL 中 SparkSession 是创建 DataFrames 和执行 SQL 的入口
创建 DataFrames 有三种方式：

- (1) 从一个已存在的 RDD 进行转换
- (2) 从 JSON/Parquet/CSV/ORC 等结构化文件源创建
- (3) 从 Hive/JDBC 各种外部结构化数据源（服务）创建

核心要义：创建 DataFrame，需要创建 “RDD + 元信息 schema 定义” + 执行计划

rdd 来自于数据

schema 则可以由开发人员定义，或者由框架从数据中推断

3.2.1 从结构化文件创建 DF

(1) 从 csv 文件（不带 header）进行创建

csv 文件内容：

```
1,张飞,21,北京,80.0
2,关羽,23,北京,82.0
3,赵云,20,上海,88.6
4,刘备,26,上海,83.0
5,曹操,30,深圳,90.0
```

代码示例：

```
val df = spark.read.csv("data_ware/demodata/stu.csv")
df.printSchema()
df.show()
```

结果如下：

```
root
|-- _c0: string (nullable = true)
|-- _c1: string (nullable = true)
|-- _c2: string (nullable = true)
|-- _c3: string (nullable = true)
|-- _c4: string (nullable = true)
```

```
+---+---+---+---+---+
|_c0|_c1|_c2|_c3|_c4|
+---+---+---+---+---+
|  1| 张飞| 21| 北京|80.0|
|  2| 关羽| 23| 北京|82.0|
|  3| 赵云| 20| 上海|88.6|
|  4| 刘备| 26| 上海|83.0|
|  5| 曹操| 30| 深圳|90.0|
+---+---+---+---+---+
```

可以看出，框架对读取进来的 csv 数据，自动生成的 schema 中，

字段名为: _c0, _c1,

字段类型全为 String

不一定是符合我们需求的

(2)从 csv 文件（不带 header）自定义 Schema 进行创建

```
// 创建 DataFrame 时，传入自定义的 schema
// schema 在 api 中用 StructType 这个类来描述，字段用 StructField 来描述
val schema = new StructType()
    .add("id", DataTypes.IntegerType)
    .add("name", DataTypes.StringType)
    .add("age", DataTypes.IntegerType)
    .add("city", DataTypes.StringType)
    .add("score", DataTypes.DoubleType)

val df = spark.read.schema(schema).csv("data_ware/demodata/stu.csv")
df.printSchema()
df.show()
```

Schema 信息:

```
root
|-- id: integer (nullable = true)
|-- name: string (nullable = true)
|-- age: integer (nullable = true)
|-- city: string (nullable = true)
|-- score: double (nullable = true)
```

数据信息:

```
+---+-----+---+-----+
| id|name  |age|city  |score|
+---+-----+---+-----+
| 1| 张飞| 21| 北京| 80.0|
| 2| 关羽| 23| 北京| 82.0|
| 3| 赵云| 20| 上海| 88.6|
| 4| 刘备| 26| 上海| 83.0|
| 5| 曹操| 30| 深圳| 90.0|
+---+-----+---+-----+
```

(3)从 csv 文件（带 header）进行创建

csv 文件内容:

```
id,name,age,city,score
1,张飞,21,北京,80.0
2,关羽,23,北京,82.0
3,赵云,20,上海,88.6
4,刘备,26,上海,83.0
5,曹操,30,深圳,90.0
```

注意：此文件的第一行是字段描述信息，需要特别处理，否则会被当做 rdd 中的一行数据

代码示例:

关键点—设置一个 header=true 的参数

```
val df = spark.read.option("header",true).csv("data_ware/demodata/stu.csv")
df.printSchema()
df.show()
```

结果如下:

```
root
|-- id: string (nullable = true)
|-- name: string (nullable = true)
```

```
-- age: string (nullable = true)
-- city: string (nullable = true)
-- score: string (nullable = true)

+---+---+---+---+
| id|name|age|city|score|
+---+---+---+---+
| 1| 张飞| 21| 北京| 80.0|
| 2| 关羽| 23| 北京| 82.0|
| 3| 赵云| 20| 上海| 88.6|
| 4| 刘备| 26| 上海| 83.0|
| 5| 曹操| 30| 深圳| 90.0|
+---+---+---+---+
```

问题：虽然字段名正确指定，但是字段类型还是无法确定，默认情况下全视作 `String` 对待，当然，可以开启一个参数 `inferSchema=true` 来让框架对 `csv` 中的数据字段进行合理的类型推断

```
val df = spark.read
  .option("header",true)
  .option("inferSchema",true)
  .csv("data_ware/demodata/stu.csv")
df.printSchema()
df.show()
```

如果推断的结果不如人意，当然可以指定自定义 `schema`
让框架自动推断 `schema`，效率低不建议！

(4)从 JSON 文件进行创建

准备 json 数据文件

```
{"name": "Michael"}
{"name": "Andy", "age": 30}
{"name": "Justin", "age": 19}
```

代码示例

```
val df = spark.read/*.schema(schema)*/.json("data_ware/demodata/people.json")
df.printSchema()
df.show()
```

(5)从 Parquet 文件进行创建

Parquet 文件是一种列式存储文件格式，文件自带 schema 描述信息

准备测试数据

任意拿一个 dataframe，调用 write.parquet()方法即可将 df 保存为一个 parquet 文件

代码示例

```
val df = spark.read.parquet("data/parquet/")
```

(6)从 orc 文件进行创建

代码示例

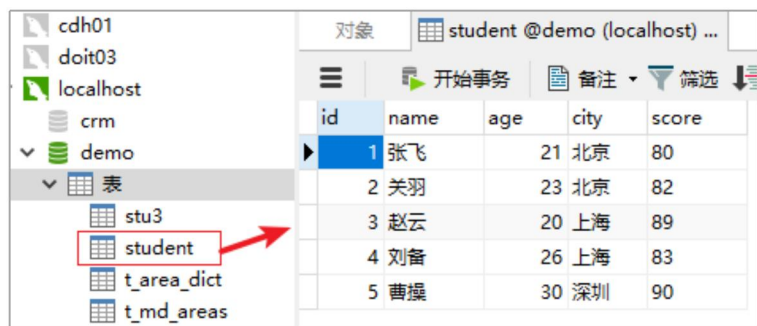
```
val df = spark.read.orc("data/orcfiles/")
```

3.2.3 外部存储服务创建 DF

(1) 从 JDBC 连接数据库服务器进行创建

实验准备

在一个 mysql 服务器中，创建一个数据库 demo，创建一个表 student，如下：



对象	student @demo (localhost) ...
	开始事务 备注 筛选
	id name age city score
	1 张飞 21 北京 80
	2 关羽 23 北京 82
	3 赵云 20 上海 89
	4 刘备 26 上海 83
	5 曹操 30 深圳 90

注：要使用 jdbc 连接读取数据库的数据，需要引入 jdbc 的驱动 jar 包依赖

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>5.1.47</version>
</dependency>
```

代码示例

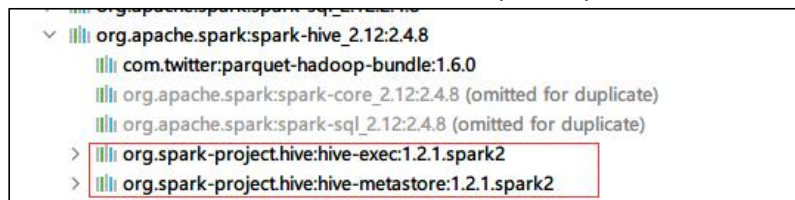
```
val props = new Properties()
props.setProperty("user", "root")
props.setProperty("password", "root")
val df = spark.read.jdbc("jdbc:mysql://localhost:3306/demo", "student", props)
df.show()
```

结果如下：

```
+---+-----+---+-----+
| id|name  |age|city  |score|
+---+-----+---+-----+
| 1| 张飞 | 21| 北京 | 80.0 |
| 2| 关羽 | 23| 北京 | 82.0 |
| 3| 赵云 | 20| 上海 | 88.6 |
| 4| 刘备 | 26| 上海 | 83.0 |
| 5| 曹操 | 30| 深圳 | 90.0 |
+---+-----+---+-----+
```

(1) SparkSql 整合 Hive 的原理

SparkSql 添加了 spark-hive 的依赖，并在 sparkSession 构造时开启 enableHiveSupport 后，就整合了 hive 的功能（通俗说，就是 sparksql 具备了 hive 的功能）：



既然具备了 hive 的功能，那么就可以执行一切 hive 中能执行的动作：

- 建表
- show 表

- 建库
- show 库
- alter 表
-

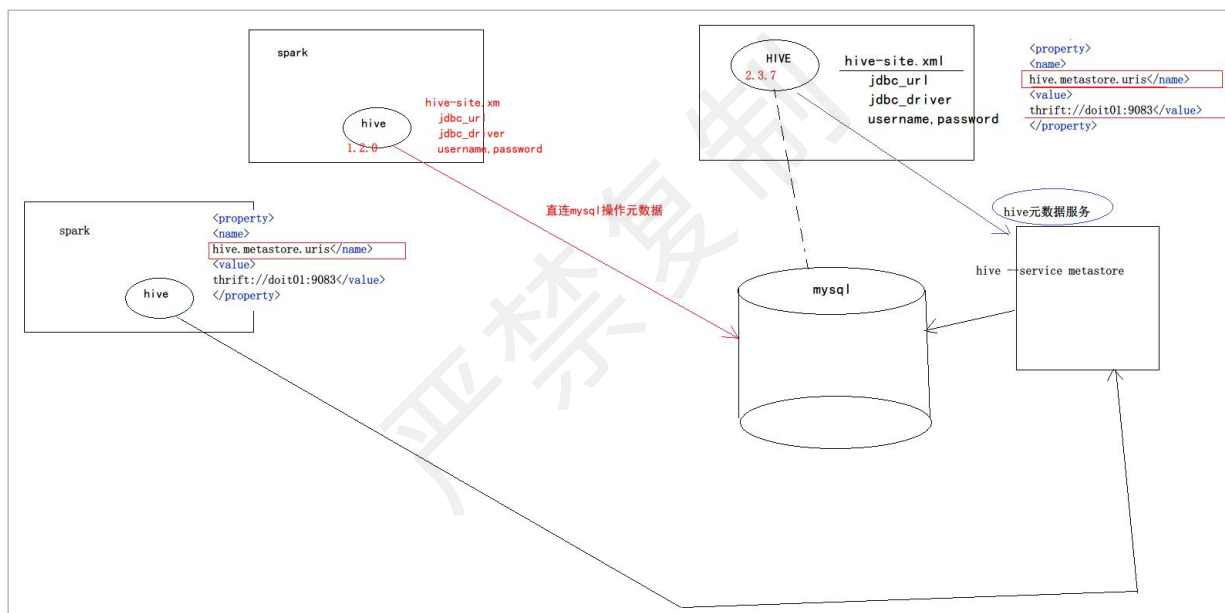
只不过，此时看见的表是 spark 中集成的 hive 的本地元数据库中的表！

如果想让 spark 中集成的 hive，看见你外部集群中的 hive 的表，只要修改配置：把 spark 端的 hive 的元数据服务地址，指向外部集群中 hive 的元数据服务地址；

有两种指定办法：

- 在 spark 端加入 hive-site.xml，里面配置 目标元数据库 mysql 的连接信息
这会使得 spark 中集成的 hive 直接访问 mysql 元数据库

- 在 spark 端加入 hive-site.xml，里面配置 目标 hive 的元数据服务器地址
这会使得 spark 中集成的 hive 通过外部独立的 hive 元数据服务来访问元数据库



(3) 从 Hive 创建 DataFrame

Sparksql 通过 spark-hive 整合包，来集成 hive 的功能

Sparksql 加载“外部独立 hive”的数据，本质上是不需要“外部独立 hive”参与的，因为“外部独立 hive”的表数据就在 hdfs 中，元数据信息在 mysql 中

不管数据还是元数据，sparksql 都可以直接去获取！

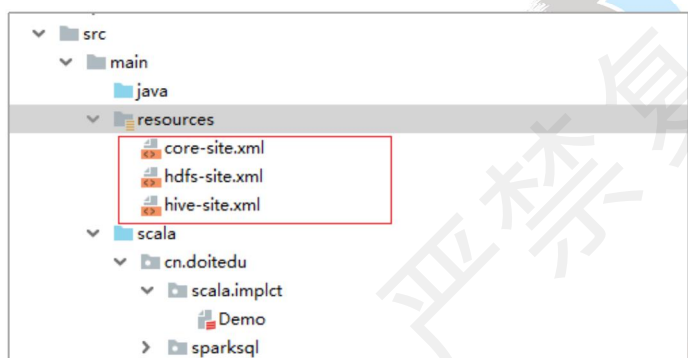
步骤:

1) 要在工程中添加 spark-hive 的依赖 jar 以及 mysql 的 jdbc 驱动 jar

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-hive_2.11</artifactId>
  <version>2.4.4</version>
</dependency>

<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>5.1.48</version>
</dependency>
```

2) 要在工程中添加 hive-site.xml/core-site.xml 配置文件



3) 创建 sparksession 时需要调用.enableHiveSupport()方法

```
val spark = SparkSession
  .builder()
  .appName(this.getClass.getSimpleName)
  .master("local[*]")
  // 启用 hive 支持,需要调用 enableHiveSupport, 还需要添加一个依赖 spark-hive
  // 默认 sparksql 内置了自己的 hive
  // 如果程序能从 classpath 中加载到 hive-site 配置文件, 那么它访问的 hive 元数据库就不是本地内置的了,
  // 而是配置中所指定的元数据库了
  // 如果程序能从 classpath 中加载到 core-site 配置文件, 那么它访问的文件系统也不再是本地文件系统了,
  // 而是配置中所指定的 hdfs 文件系统了
```

```
.enableHiveSupport()  
.getOrCreate()
```

4) 加载 hive 中的表

```
val df = spark.sql("select * from t1")
```

注意点: 如果自己也用 `dataframe` 注册了一个同名的视图, 那么这个视图名会替换掉 `hive` 的表

(4) 从 Hbase 创建 DataFrame

其实, `sparksql` 可以连接任意外部数据源 (只要有对应的“连接器”即可)
`Sparksql` 对 `hbase` 是有第三方连接器 (华为) 的, 但是久不维护!

建议用 `hive` 作为连接器 (hive 可以访问 `hbase`, 而 `sparksql` 可以集成 `hive`)

- 在 `hbase` 中建表

```
create 'doitedu_stu', 'f'
```

- 插入数据到 `hbase` 表

```
put 'doitedu_stu', '001', 'f:name', 'zhangsan'  
put 'doitedu_stu', '001', 'f:age', '26'  
put 'doitedu_stu', '001', 'f:gender', 'm'  
put 'doitedu_stu', '001', 'f:salary', '28000'  
put 'doitedu_stu', '002', 'f:name', 'lisi'  
put 'doitedu_stu', '002', 'f:age', '22'  
put 'doitedu_stu', '002', 'f:gender', 'm'  
put 'doitedu_stu', '002', 'f:salary', '26000'  
put 'doitedu_stu', '003', 'f:name', 'wangwu'  
put 'doitedu_stu', '003', 'f:age', '21'  
put 'doitedu_stu', '003', 'f:gender', 'f'  
put 'doitedu_stu', '003', 'f:salary', '24000'  
put 'doitedu_stu', '004', 'f:name', 'zhaoliu'  
put 'doitedu_stu', '004', 'f:age', '22'  
put 'doitedu_stu', '004', 'f:gender', 'f'  
put 'doitedu_stu', '004', 'f:salary', '25000'
```

- 创建 hive 外部表映射 hbase 中的表

```
CREATE EXTERNAL TABLE doitedu_stu
(
  id      string ,
  name    string ,
  age     int    ,
  gender  string ,
  salary  double
)
STORED BY 'org.apache.hadoop.hive.hbase.HBaseStorageHandler'
WITH SERDEPROPERTIES ( 'hbase.columns.mapping'=':key,f:name,f:age,f:gender,f:salary')
TBLPROPERTIES ( 'hbase.table.name'='default:doitedu_stu')
;
```

- 工程中放置 hbase-site.xml 配置文件

```
<configuration>
  <property>
    <name>hbase.rootdir</name>
    <value>hdfs://doit01:8020/hbase</value>
  </property>

  <property>
    <name>hbase.cluster.distributed</name>
    <value>true</value>
  </property>

  <property>
    <name>hbase.zookeeper.quorum</name>
    <value>doit01:2181,doit02:2181,doit03:2181</value>
  </property>

  <property>
    <name>hbase.unsafe.stream.capability.enforce</name>
    <value>false</value>
  </property>
</configuration>
```

- 工程中添加 `hive-hbase-handler` 连接器依赖

```
<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-hbase-handler</artifactId>
  <version>2.3.7</version>
  <exclusions>
    <exclusion>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-common</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

- 以读取 hive 表的方式直接读取即可

```
spark.sql("select * from doitedu_stu")
```

(5) RDD 方式操作 Elastic Search

详细参考: [Apache Spark support | Elasticsearch for Apache Hadoop \[master\] | Elastic](#)

1) 添加依赖

```
<!-- https://mvnrepository.com/artifact/org.elasticsearch/elasticsearch-spark-20 -->
<dependency>
  <groupId>org.elasticsearch</groupId>
  <artifactId>elasticsearch-spark-20_2.12</artifactId>
  <version>7.14.1</version>
</dependency>
```

2) 从 es 读取数据为 rdd

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._

import org.elasticsearch.spark._

...

val conf = ...
val sc = new SparkContext(conf)

val RDD = sc.esRDD("radio/artists")
```

3) 将 rdd 写入 es

```
import org.apache.spark.SparkContext
import org.elasticsearch.spark.rdd.EsSpark

// define a case class
case class Trip(departure: String, arrival: String)

val upcomingTrip = Trip("OTP", "SFO")
val lastWeekTrip = Trip("MUC", "OTP")

val rdd = sc.makeRDD(Seq(upcomingTrip, lastWeekTrip))
EsSpark.saveToEs(rdd, "spark/docs")
```

或者直接使用 EsSpark 来实现数据写入

```
EsSpark.saveToEs(rdd, "spark/docs", Map("es.mapping.id" -> "id"))
```

(2) Dataframe 方式操作 Elastic Search

1) 读取 ES

```
import org.apache.spark.sql.SQLContext

import org.elasticsearch.spark.sql._
```

```
...

val sql = new SQLContext(sc)

val people = sql.esDF("spark/people")

// check the associated schema
println(people.schema.treeString)
// root
// |-- name: string (nullable = true)
// |-- surname: string (nullable = true)
// |-- age: long (nullable = true)
```

2) 写入 ES

```
// reusing the example from Spark SQL documentation

import org.apache.spark.sql.SQLContext
import org.apache.spark.sql.SQLContext._

import org.elasticsearch.spark.sql._

...

// sc = existing SparkContext
val sqlContext = new SQLContext(sc)

// case class used to define the DataFrame
case class Person(name: String, surname: String, age: Int)

// create DataFrame
val people = sc.textFile("people.txt")
    .map(_._split(","))
    .map(p => Person(p(0), p(1), p(2).trim.toInt))
    .toDF()

people.saveToEs("spark/people")
```

3.2 输出 DF 的各种方式

(1) 展现在控制台

```
df.show()  
df.show(10)  
df.show(10,false) // 不要截断列
```

(2) 保存为文件

```
/**  
 * @date: 2019/10/11  
 * @site: www.doitedu.cn  
 * @author: hunter.d 涛哥  
 * @qq: 657270652  
 * @description:  
 *   保存 dataframe  
 */  
object Demo17_SaveDF {  
  
  def main(args: Array[String]): Unit = {  
  
    val spark = SparkUtil.getSpark()  
  
    val df = spark.read.option("header",true).csv("data/stu2.csv")  
  
    val res = df.where("id>3").select("id","name")  
  
    // 展示结果  
    res.show(10,false)  
  
    // 保存结果为文件:  parquet,json,csv,orc,textfile  
    /*res.write.parquet("d:/retest/parquetfile/")  
  
    res.write.csv("d:/retest/csvfile")  
  
    res.write.orc("d:/retest/orcfile")  
  
    res.write.json("d:/retest/jsonfile")*/
```

```
// 文本文件是自由格式，框架无法判断该输出什么样的形式
// 要将 df 输出为普通文本文件，则需要将 df 变成一个列
// res.write.text("d:/retest/textfile")
res.selectExpr("concat_ws('\001',id,name)")
  .write.text("d:/retest/textfile")

spark.close()

}
}
```

(3) 保存到 RDBMS

将 dataframe 写入 mysql 的表

```
// 将 dataframe 通过 jdbc 写入 mysql
val props = new Properties()
props.setProperty("user", "root")
props.setProperty("password", "root")

// 可以通过 SaveMode 来控制写入模式: SaveMode.Append/Ignore/Overwrite/ErrorIfExists(默认)
res.write.mode(SaveMode.Append).jdbc("jdbc:mysql://localhost:3306/demo?characterEncoding=utf8", "res", props)
```

将 dataframe 写入 oracle

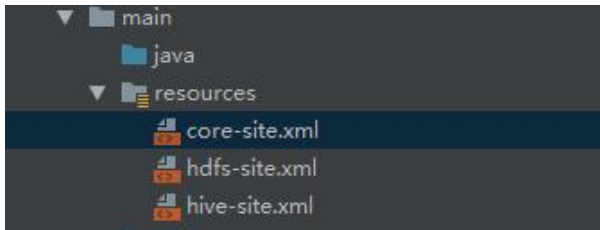
(4) 写入 hive

开启 spark 的 hive 支持

```
val spark = SparkSession
  .builder()
  .appName("")
  .master("local[*]")
```

```
.enableHiveSupport()  
.getOrCreate()
```

放入配置文件



导入 hive 的支持依赖

```
<dependency>  
  <groupId>org.apache.spark</groupId>  
  <artifactId>spark-hive_2.11</artifactId>  
  <version>2.4.4</version>  
</dependency>  
  
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>  
  <version>5.1.48</version>  
</dependency>
```

写代码:

```
// 将 dataframe 写入 hive, saveAsTable 就是保存为 hive 的表  
// 前提, spark 要开启 hiveSupport 支持, spark-hive 的依赖, hive 的配置文件  
res.write.saveAsTable("res")
```

(5) 写入 NoSql 数据库

写入 hbase

写入 elastic search

(6) DF 输出时的分区操作

Hive 中对表数据的存储，可以将数据分为多个子目录！
比如：

```
create table tx(id int,name string) partitioned by (city string);
load data inpath '/data/1.dat' into table tx partition(city="beijing")
load data inpath '/data/2.dat' into table tx partition(city="shanghai")
....
```

Hive 的表 tx 的目录结构如下：

```
/user/hive/warehouse/tx/
    city=beijing/1.dat
    city=shanghai/2.dat
```

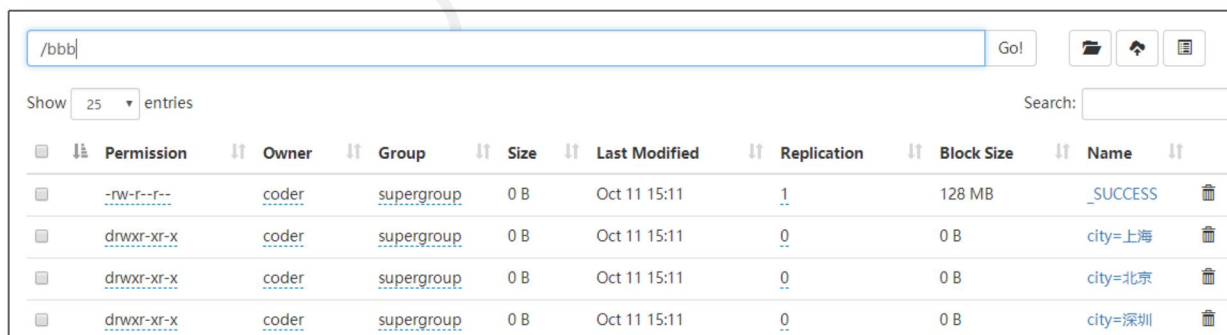
查询的时候，分区标识字段，可以看做表的一个字段来用

```
Select * from tx where city='shanghai'
```

那么，sparksql 既然是跟 hive 兼容的，必然也有对分区存储支持的机制！

1. 能识别解析分区

有如下数据结构形式：



The screenshot shows a file browser interface with a search bar containing '/bbb'. Below the search bar, it says 'Show 25 entries'. A table lists the files and directories in the current view. The table has columns for Permission, Owner, Group, Size, Last Modified, Replication, Block Size, and Name. The files listed are:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	coder	supergroup	0 B	Oct 11 15:11	1	128 MB	_SUCCESS
drwxr-xr-x	coder	supergroup	0 B	Oct 11 15:11	0	0 B	city=上海
drwxr-xr-x	coder	supergroup	0 B	Oct 11 15:11	0	0 B	city=北京
drwxr-xr-x	coder	supergroup	0 B	Oct 11 15:11	0	0 B	city=深圳

/**

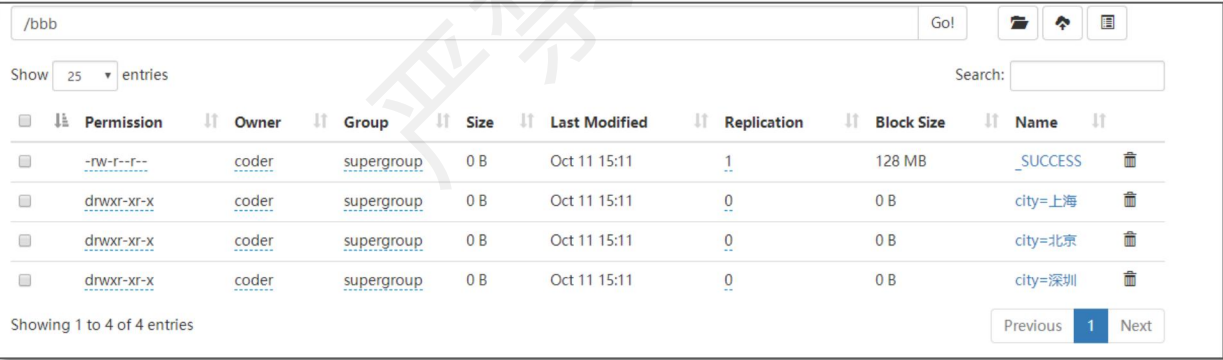
- * sparksql 对分区机制的支持
- * 识别已存在分区结构 /aaa/city=a; /aaa/city=b;
- * 会将所有子目录都理解为数据内容，会将子目录中的 city 理解为一个字段

```
*/  
spark.read.csv("/aaa").show()
```

2. 能将数据按分区机制输出

```
/**  
 * sparksql 对分区机制的支持  
 * 将 dataframe 存储为分区结构  
 */  
val dfp = spark.read.option("header",true).csv("data/stu2.csv")  
dfp.write.partitionBy("city").csv("/bbb")  
  
/**  
 * 将数据分区写入 hive  
 * 注意：写入 hive 的默认文件格式是 parquet  
 */  
dfp.write.partitionBy("sex").saveAsTable("res_p")
```

输入结构如下：



Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	coder	supergroup	0 B	Oct 11 15:11	1	128 MB	_SUCCESS
drwxr-xr-x	coder	supergroup	0 B	Oct 11 15:11	0	0 B	city=上海
drwxr-xr-x	coder	supergroup	0 B	Oct 11 15:11	0	0 B	city=北京
drwxr-xr-x	coder	supergroup	0 B	Oct 11 15:11	0	0 B	city=深圳

3.3 DF 数据运算操作

3.3.1 纯 SQL 操作

核心要义：将 DataFrame 注册为一个临时视图 view，然后就可以针对 view 直接执行各种 sql

临时视图有两种：session 级别视图，global 级别视图；

session 级别视图是 Session 范围内有效的，Session 退出后，表就失效了；

全局视图则在 application 级别有效；

注意使用全局表时需要全路径访问：global_temp.people

```
// application 全局有效
df.createGlobalTempView("stu")
spark.sql(
  """
  |select * from global_temp.stu a order by a.score desc
  """).stripMargin()
  .show()

// session 有效
df.createTempView("s")
spark.sql(
  """
  |select * from s order by score
  """).stripMargin()
  .show()

val spark2 = spark.newSession()
// 全局有效的 view 可以在 session2 中访问
spark2.sql("select id,name from global_temp.stu").show()
// session 有效的 view 不能在 session2 中访问
spark2.sql("select id,name from s").show()
```

以上只是对语法的简单示例，可以扩展到任意复杂的 sql

挑战一下？

求出每个城市中，分数最高的学生信息；

Go go go !

3.3.2 DSL 风格 API(TableApi)语法

DSL 风格 API，就是用编程 api 的方式，来实现 sql 语法

DSL：特定领域语言

dataset 的 tableApi 有一个特点：运算后返回值必回到 dataframe

因为 select 后，得到的结果，无法预判返回值的具体类型，只能用通用的 Row 封装

数据准备

```
val df = spark.read
  .option("header", true)
  .option("inferSchema", true)
  .csv("data_ware/demodata/stu.csv")
```

(1) 基本 select 及表达式

```
/**
 * 逐行运算
 */
// 使用字符串表达"列"
df.select("id", "name").show()

// 如果要用字符串形式表达 sql 表达式，应该使用 selectExpr 方法
df.selectExpr("id+1", "upper(name)").show
// select 方法中使用字符串 sql 表达式，会被视作一个列名从而出错
// df.select("id+1", "upper(name)").show()

import spark.implicits._
// 使用$符号创建 Column 对象来表达"列"
df.select($"id", $"name").show()

// 使用单边单引号创建 Column 对象来表达"列"
df.select('id, 'name).show()

// 使用 col 函数来创建 Column 对象来表达"列"
import org.apache.spark.sql.functions._
df.select(col("id"), col("name")).show()

// 使用 Dataframe 的 apply 方法创建 Column 对象来表达列
df.select(df("id"), df("name")).show()
```

```
// 对 Column 对象直接调用 Column 的方法，或调用能生成 Column 对象的 functions 来实现 sql 中的运算表达式
df.select('id.plus(2).leq("4").as("id2"),upper('name')).show()
df.select('id+2 <= 4 as "id2",upper('name')).show()
```

(3) 字段重命名

```
/**
 * 字段重命名
 */
// 对 column 对象调用 as 方法
df.select('id as "id2",$"name".as("n2"),col("age") as "age2").show()

// 在 selectExpr 中直接写 sql 的重命名语法
df.selectExpr("cast(id as string) as id2","name","city").show()

// 对 dataframe 调用 withColumnRenamed 方法对指定字段重命名
df.select("id","name","age").withColumnRenamed("id","id2").show()

// 对 dataframe 调用 toDF 对整个字段名全部重设
df.toDF("id2","name","age","city2","score").show()
```

(2) 条件过滤

```
/**
 * 逐行过滤
 */
df.where("id>4 and score>95")
df.where('id > 4 and 'score > 95').select("id","name","age").show()
```

(4) 分组聚合

```
/**
 * 分组聚合
 */
df.groupBy("city").count().show()
```

```
df.groupBy("city").min("score").show()
df.groupBy("city").max("score").show()
df.groupBy("city").sum("score").show()
df.groupBy("city").avg("score").show()

df.groupBy("city").agg(("score", "max"), ("score", "sum")).show()
df.groupBy("city").agg("score" -> "max", "score" -> "sum").show()
```

(4) 子查询

```
/**
 * 子查询
 * 相当于:
 * select
 * *
 * from
 * (
 *   select
 *   city,sum(score) as score
 *   from stu
 *   group by city
 * ) o
 * where score>165
 */
df.groupBy("city")
  .agg(sum("score") as "score")
  .where("score > 165")
  .select("city", "score")
  .show()
```

(5) Join 关联查询

```
package cn.doitedu.sparksql

import org.apache.spark.sql.{DataFrame, SparkSession}

/**
 * @date: 2019/10/9
```

```
* @site: www.doitedu.cn
* @author: hunter.d 涛哥
* @qq: 657270652
* @description:
* 用 DSL 风格 api 来对 dataframe 进行运算
*/

object Demo14_DML_DSLapi {
  def main(args: Array[String]): Unit = {

    val spark = SparkUtil.getSpark()
    import spark.implicits._

    val df = spark.read.option("header",true).csv("data/stu2.csv")
    val df2 = spark.read.option("header",true).csv("data/stu22.csv")

    /**
     * SQL 中都有哪些运算?
     * 1. 查询字段 (id)
     * 2. 查询表达式 (算术运算, 函数 age+10, upper(name))
     * 3. 过滤
     * 4. 分组聚合
     * 5. 子查询
     * 6. 关联查询
     * 7. union 查询
     * 8. 窗口分析
     * 9. 排序
     */

    //selectOp(spark,df)
    //whereOp(spark,df)
    //groupByOp(spark,df)
    joinOp(spark,df,df2)

    spark.close()
  }

  /**
   * 关联查询
   * @param spark
   * @param df1
   */
  def joinOp(spark:SparkSession,df1:DataFrame,df2:DataFrame): Unit ={
```

```
// 笛卡尔积
//df1.crossJoin(df2).show()

// 给 join 传入一个连接条件： 这种方式，要求，你的 join 条件字段在两个表中都存在且同名
df1.join(df2,"id").show()

// 传入多个 join 条件列，要求两表中这多个条件列都存在且同名
df1.join(df2,Seq("id","sex")).show()

// 传入一个自定义的连接条件表达式
df1.join(df2,df1("id") + 1 === df2("id")).show()

// 还可以传入 join 方式类型： inner(默认), left ,right, full ,left_semi, left_anti
df1.join(df2,df1("id")+1 === df2("id"),"left").show()
df1.join(df2,Seq("id"),"right").show()

/**
 * 总结：
 * join 方式：   joinType: String
 * join 条件：
 * 可以直接传 join 列名： usingColumn/usingColumns : Seq(String) 注意：右表的 join 列数据不会出
现结果中
 * 可以传 join 自定义表达式： Column.+(1) === Column      df1("id")+1 === df2("id")
 */
}
```

(6) Union 操作

Sparksql 中的 union，其实是 union all

```
df.union(df).show()
```

(7) 窗口分析函数调用

测试数据:

```
id,name,age,sex,city,score
1,张飞,21,M,北京,80
2,关羽,23,M,北京,82
7,周瑜,24,M,北京,85
3,赵云,20,F,上海,88
4,刘备,26,M,上海,83
8,孙权,26,M,上海,78
5,曹操,30,F,深圳,90.8
6,孔明,35,F,深圳,77.8
9,吕布,28,M,深圳,98
```

求每个城市中成绩最高的两个人的信息, 如果用 sql 写:

```
select
id,name,age,sex,city,score
from
(
    select
        id,name,age,sex,city,score,
        row_number() over(partition by city order by score desc) as rn
    from t
) o
where rn<=2
```

DSL 风格的 API 实现:

```
package cn.doitedu.sparksql

import org.apache.spark.sql.expressions.Window

/**
 * @date: 2019/10/11
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description:
```

```
* 用 dsl 风格 api 实现 sql 中的窗口分析函数
*/
object Demo15_DML_DSLAPI_WINDOW {

  def main(args: Array[String]): Unit = {

    val spark = SparkUtil.getSpark()

    val df = spark.read.option("header", true).csv("data/stu2.csv")

    import spark.implicits._
    import org.apache.spark.sql.functions._

    val window = Window.partitionBy('city').orderBy('score.desc')

    df.select('id', 'name', 'age', 'sex', 'city', 'score', row_number().over(window) as "rn")
      .where('rn <= 2)
      .drop("rn") // 最后结果中不需要 rn 列，可以 drop 掉这个列
      // .select('id', 'name', 'age', 'sex', 'city', 'score') // 或者用 select 指定你需要的列
      .show()

    spark.close()
  }
}
```

3.3.3 DataFrame/dataset 的 RDD 风格算子

Dataset 提供与 RDD 类似的编程算子，即 map/flatMap/reduceByKey 等等，不过本方式使用略少：

- 如果方便使用 sql 表达的逻辑，首选 sql
- 如果不方便使用 sql 表达，则可以把 Dataset 转成 RDD 后使用 RDD 的算子

直接在 Dataset 上调用类似 RDD 风格算子的代码示例如下：

```
/**
 * 四、 dataset/dataframe 调 RDD 算子
 *
 * dataset 调 rdd 算子，返回的还是 dataset[U]，不过需要对应的 Encoder[U]
 *
 */
val ds4: Dataset[(Int, String, Int)] = ds2.map(p => (p.id, p.name, p.age + 10)) // 元组有隐式 Encoder 自动传入
val ds5: Dataset[JavaPerson] = ds2.map(p => new
JavaPerson(p.id, p.name, p.age * 2)) (Encoders.bean(classOf[JavaPerson])) // Java 类没有自动隐式 Encoder，
需要手动传

val ds6: Dataset[Map[String, String]] = ds2.map(per => Map("name" -> per.name, "id" -> (per.id + ""),
"age" -> (per.age + "")))
ds6.printSchema()
/**
 * root
 * |-- value: map (nullable = true)
 * |   |-- key: string
 * |   |-- value: string (valueContainsNull = true)
 */
// 从 ds6 中查询每个人的姓名
ds6.selectExpr("value['name']")

// dataframe 上调 RDD 算子，就等价于 dataset[Row]上调 rdd 算子
val ds7: Dataset[(Int, String, Int)] = frame.map(row=>{
    val id: Int = row.getInt(0)
    val name: String = row.getAs[String]("name")
    val age: Int = row.getAs[Int]("age")
    (id, name, age)
})
```

```
// 利用模式匹配从 row 中抽取字段数据
val ds8: Dataset[Per] = frame.map({
  case Row(id:Int,name:String,age:Int) => Per(id,name,age*10)
})
```

3.5 Dataset 与 RDD 混编

3.5.1 DataSet 和 Dataframe 的区别

狭义上，Dataset 中装的是用户自定义的类型

那么在抽取数据时，比较方便 `stu.id` 且类型会得到编译时检查

狭义上，dataframe 中装的是 Row（框架内置的一个通用类型）

那么在抽取数据时，不太方便，得通过脚标，或者字段名，而且还得强转

```
val x:Any = row.get(1)
val x:Double = row.getDouble(1)
val x:Double = row.getAs[Double]("salary")
```

```
/**
 * dataset 存在的意义？
 * 意义要从它的特点说起：
 * ds 的特点是，可以存储各种自定义类型，自定义类型中，各字段是有类型约束（所以 ds 是强类型约束的）
 * df 只能存储 row 类型，而 row 类型中的字段是没有“类型约束”，全是 any（所以 df 是弱类型约束的）
 */
ds.map(bean => {
  // val id:String = bean.id // 提取数据时不会产生类型匹配错误，编译时就会检查
})

val _df: Dataset[Row] = ds.toDF()
_df.map(row => {
  val name: Int = row.getInt(1) // 明明类型不匹配，但是编译时无法检查，运行时才会抛异常
})
```

3.5.3 DataFrame/dataset 转成 RDD 后取数

要义：有些运算场景下，通过 SQL 语法实现计算逻辑比较困难，可以将 DataFrame 转成 RDD 算子来操作，而 DataFrame 中的数据是以 RDD[Row] 类型封装的，因此，要对 DataFrame 进行 RDD 算子操作，只需要掌握如何从 Row 中解构出数据即可

示例数据 stu.csv

```
id,name,age,city,score
1,张飞,21,北京,80.0
2,关羽,23,北京,82.0
3,赵云,20,上海,88.6
4,刘备,26,上海,83.0
5,曹操,30,深圳,90.0
```

(1) 从 Row 中取数方式 1: 索引号

```
val df = spark.read
  .option("header", true)
  .option("inferSchema", true)
  .csv(path = "data_ware/demodata/stu.csv")

// 获取dataframe中的rdd
val rdd: RDD[Row] = df.rdd
rdd.map(row => {
  row.get
})
```

示例代码

```
val rdd: RDD[Row] = df.rdd
rdd.map(row => {
  val id = row.get(0).asInstanceOf[Int]
  val name = row.getString(1)
  (id, name)
}).take(10).foreach(println)
```

(2) 从 Row 中取数方式 2：字段名

```
rdd.map(row=>{
    val id = row.getAs[Int]("id")
    val name = row.getAs[String]("name")
    val age = row.getAs[Int]("age")
    val city = row.getAs[String]("city")
    val score = row.getAs[Double]("score")
    (id,name,age,city,score)
}).take(10).foreach(println)
```

(3) 从 Row 中取数方式 3：模式匹配

```
rdd.map({
    case Row(id: Int, name: String, age: Int, city: String, score: Double)
=> {
    // do anything
    (id,name,age,city,score)
    }
}).take(10).foreach(println)
```

(4) 完整示例

```
package cn.doitedu.sparksql

import org.apache.spark.rdd.RDD
import org.apache.spark.sql.{Dataset, Row}
import org.apache.spark.sql.types.{DoubleType, IntegerType, StringType, StructField, StructType}

/**
 * @date: 2019/10/11
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description:
 * 有些场景下，逻辑不太方便使用 sql 去实现，可能需要将 dataframe 退化成 RDD 来计算
 * 示例需求： 求每种性别的成绩总和
 */
```

```
*/  
object Demo16_DML_RDD {  
  def main(args: Array[String]): Unit = {  
  
    val spark = SparkUtil.getSpark()  
    // val schema = new  
    StructType(Array(StructField("id", IntegerType), StructField("name", StringType)))  
  
    // val schema = new StructType((StructField("id", IntegerType)::  
    StructField("name", StringType) :: Nil).toArray)  
  
    val schema = new StructType()  
      .add("id", IntegerType)  
      .add("name", StringType)  
      .add("age", IntegerType)  
      .add("sex", StringType)  
      .add("city", StringType)  
      .add("score", DoubleType)  
  
    val df = spark.read.schema(schema).option("header", true).csv("data/stu2.csv")  
  
    // 可以直接在 dataframe 上用 map 等 rdd 算子  
    // 框架会把算子返回的结果 RDD 再转回 dataset，需要一个能对 RDD[T] 进行解析的 Encoder[T] 才行  
    // 好在大部分 T 类型都可以有隐式的 Encoder 来支持  
    import spark.implicits._  
    val ds2: Dataset[(Int, String)] = df.map(row=>{  
      val id = row.getAs[Int]("id")  
      val name = row.getAs[String]("name")  
      (id, name)  
    })  
  
    // dataframe 中取出 rdd 后，就是一个 RDD[Row]  
    val rd: RDD[Row] = df.rdd  
    // 从 Row 中取数据，就可以变成任意你想要的类型  
    val rdd2: RDD[(Int, String, Int, String, String, Double)] = rd.map(row=>{  
  
      // dataframe 是一种弱类型结构（在编译时无法检查类型，因为数据被装在了一个 array[any] 中）  
      // val id = row.getDouble(1) // 如果类型取错，编译时是无法检查的，运行时才会报错  
  
      // 可以根据字段的脚标去取  
      val id: Int = row.getInt(0)  
      val name: String = row.getString(1)
```

```
val age: Int = row.getAs[Int](2)

// 可以根据字段名称去取
val sex: String = row.getAs[String]("sex")
val city: String = row.getAs[String]("city")
val score: Double = row.getAs[Double]("score")

(id,name,age,sex,city,score)
}))

/**
 * 用模式匹配从 Row 中抽取数据
 * 效果跟上面的方法是一样的，但是更简洁！
 */
val rdd22 = rd.map({
  case Row(id:Int,name:String,age:Int,sex:String,city:String,score:Double)=>{
    (id,name,age,sex,city,score)
  }
})

// 后续就跟 dataframe 没关系了，跟以前的 rdd 是一样的了
val res: RDD[(String, Double)] = rdd22.groupBy(tp=>tp._4).mapValues(iter=>{
  iter.map(_._6).sum
})

res.foreach(println)

spark.close()
}
```

3.5.4 从 RDD 创建 DataFrame

准备测试用的数据和 RDD

后续示例都起源于如下 RDD

数据文件: doit_stu.txt

1,张飞,21,北京,80.0

2,关羽,23,北京,82.0

```
3,赵云,20,上海,88.6  
4,刘备,26,上海,83.0  
5,曹操,30,深圳,90.0
```

创建 RDD:

```
val rdd:RDD[String] = spark.sparkContext.textFile("data_ware/demodata/stu.txt")
```

(1) 从 RDD[Case class 类]创建 DataFrame

注: 定义一个 case class 来封装数据, 如下, Stu 是一个 case class 类

```
val rdd:RDD[String] = spark.sparkContext.textFile("data_ware/demodata/stu.txt")
```

示例代码:

```
val rddStu: RDD[Stu] = rdd  
  // 切分字段  
  .map(_.split(","))  
  // 将每一行数据变成一个多元组 tuple  
  .map(arr => Stu(arr(0).toInt, arr(1), arr(2).toInt, arr(3), arr(4).toDouble))  
  // 创建 DataFrame  
val df = spark.createDataFrame(rddStu)  
df.show()
```

结果如下:

```
+---+-----+---+-----+  
| id|name  |age|city |score|  
+---+-----+---+-----+  
|  1| 张飞| 21| 北京| 80.0|  
|  2| 关羽| 23| 北京| 82.0|  
|  3| 赵云| 20| 上海| 88.6|  
|  4| 刘备| 26| 上海| 83.0|  
|  5| 曹操| 30| 深圳| 90.0|  
+---+-----+---+-----+
```

可以发现，框架成功地从 `case class` 的类定义中推断出了数据的 `schema`：字段类型和字段名称
Schema 获取手段：反射

当然，还有更简洁的方式，利用框架提供的隐式转换

```
// 更简洁办法
import spark.implicits._
val df = rddStu.toDF
```

(2) 从 RDD[Tuple] 创建 DataFrame

```
val rddTuple: RDD[(Int, String, Int, String, Double)] = rdd
// 切分字段
.map(_.split(","))
// 将每一行数据变换成一个多元组 tuple
.map(arr => (arr(0).toInt, arr(1), arr(2).toInt, arr(3), arr(4).toDouble))

//创建 DataFrame
val df = spark.createDataFrame(rddTuple)
df.printSchema() // 打印 schema 信息
df.show()
```

结果如下：

```
root
|-- _1: integer (nullable = false)
|-- _2: string (nullable = true)
|-- _3: integer (nullable = false)
|-- _4: string (nullable = true)
|-- _5: double (nullable = false)
```

```
+---+-----+---+-----+
| _1| _2 | _3| _4| _5 |
+---+-----+---+-----+
| 1| 张飞| 21| 北京|80.0|
| 2| 关羽| 23| 北京|82.0|
| 3| 赵云| 20| 上海|88.6|
| 4| 刘备| 26| 上海|83.0|
```

```
| 5| 曹操| 30| 深圳|90.0|  
+---+---+---+---+  
| 5| 曹操| 30| 深圳|90.0|
```

从结果中可以发现一个问题：框架从 `tuple` 元组结构中，对 `schema` 的推断，也是成功的，只是字段名是 `tuple` 中的数据访问索引。

当然，还有更简洁的方式，利用框架提供的隐式转换可以直接调用 `toDF` 创建，并指定字段名

```
// 更简洁办法  
import spark.implicits._  
val df2 = rddTuple.toDF("id","name","age","city","score")
```

（3）从 RDD[JavaBean] 创建 DataFrame

注：此处所说的 Bean，指的是用 `java` 定义的 `bean`

```
public class Stu2 {  
    private int id;  
    private String name;  
    private int age;  
    private String city;  
    private double score;  
  
    public Stu2(int id, String name, int age, String city, double score) {  
        this.id = id;  
        this.name = name;  
        this.age = age;  
        this.city = city;  
        this.score = score;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public void setId(int id) {}  
}
```

示例代码：

```
val rddBean: RDD[Stu2] = rdd
// 切分字段
.map(_.split(","))
// 将每一行数据变形形成一个 JavaBean
.map(arr => new Stu2(arr(0).toInt, arr(1), arr(2).toInt, arr(3), arr(4).toDouble))
val df = spark.createDataFrame(rddBean, classOf[Stu2])
df.show()
```

结果如下：

```
+---+-----+---+-----+
|age|city | id|name |score|
+---+-----+---+-----+
| 1| 张飞| 21| 北京|80.0|
| 2| 关羽| 23| 北京|82.0|
| 3| 赵云| 20| 上海|88.6|
| 4| 刘备| 26| 上海|83.0|
| 5| 曹操| 30| 深圳|90.0|
+---+-----+---+-----+
```

注：RDD[JavaBean]在 spark.implicit._ 中没有 toDF 的支持

（4）从 RDD[普通 Scala 类]中创建 DataFrame

注：此处的普通类指的是 scala 中定义的非 case class 的类
框架在底层将其视作 java 定义的标准 bean 类型来处理

而 scala 中定义的普通 bean 类，不具备字段的 java 标准 getters 和 setters，因而会处理失败
可以如下处理来解决

普通 scala bean 类定义：

```
class Stu3(
    @BeanProperty
    val id: Int,
    @BeanProperty
    val name: String,
    @BeanProperty
    val age: Int,
    @BeanProperty
```

```
val city: String,  
@BeanProperty  
val score: Double)
```

示例代码：

```
val rddStu3: RDD[Stu3] = rdd  
// 切分字段  
.map(_.split(","))  
// 将每一行数据变形成一个普通 Scala 对象  
.map(arr => new Stu3(arr(0).toInt, arr(1), arr(2).toInt, arr(3), arr(4).toDouble))  
val df = spark.createDataFrame(rddStu3, classOf[Stu3])  
df.show()
```

（5）从 RDD[Row] 中创建 DataFrame

注：DataFrame 中的数据，本质上还是封装在 RDD 中，而 RDD[T]总有一个 T 类型，DataFrame 内部的 RDD 中的元素类型 T 即为框架所定义的 Row 类型；

```
val rddRow = rdd  
// 切分字段  
.map(_.split(","))  
// 将每一行数据变形成一个 Row 对象  
.map(arr => Row(arr(0).toInt, arr(1), arr(2).toInt, arr(3), arr(4).toDouble))  
  
val schema = new StructType()  
.add("id", DataTypes.IntegerType)  
.add("name", DataTypes.StringType)  
.add("age", DataTypes.IntegerType)  
.add("city", DataTypes.StringType)  
.add("score", DataTypes.DoubleType)  
  
val df = spark.createDataFrame(rddRow, schema)  
df.show()
```

(6) 从 RDD[set/seq/map]中创建 DataFrame

版本 2.2.0, 新增了对 SET/SEQ 的编解码支持

版本 2.3.0, 新增了对 Map 的编解码支持

```
object Demo7_CreateDF_SetSeqMap {

  def main(args: Array[String]): Unit = {

    val spark = SparkSession.builder().appName("").master("local[*]").getOrCreate()

    val seq1 = Seq(1,2,3,4)
    val seq2 = Seq(11,22,33,44)

    val rdd: RDD[Seq[Int]] = spark.sparkContext.parallelize(List(seq1,seq2))

    import spark.implicits._
    val df = rdd.toDF()

    df.printSchema()
    df.show()

    df.selectExpr("value[0]", "size(value)").show()

    /**
     * set 类型数据 rdd 的编解码
     */
    val set1 = Set("a", "b")
    val set2 = Set("c", "d", "e")
    val rdd2: RDD[Set[String]] = spark.sparkContext.parallelize(List(set1, set2))

    val df2 = rdd2.toDF("members")
    df2.printSchema()
    df2.show()

    /**
     * map 类型数据 rdd 的编解码
     */
  }
}
```

```
val map1 = Map("father"->"mayun", "mother"->"tangyan")
val map2 = Map("father"->"huateng", "mother"->"yifei", "brother"->"sicong")
val rdd3: RDD[Map[String, String]] = spark.sparkContext.parallelize(List(map1, map2))

val df3 = rdd3.toDF("jiaren")
df3.printSchema()
df3.show()

df3.selectExpr("jiaren['mother']", "size(jiaren)", "map_keys(jiaren)", "map_values(jiaren)")
    .show(10, false)

spark.close()
}
```

set/seq 结构出来的字段类型为：array

Map 数据类型解构出来的字段类型为：map

3.5.5 从 RDD 创建 DataSet

```
import spark.implicits._

spark.createdataset|
  createDataset[T](data: Seq[T])(implicit evidence$4: Encoder[T])
  createDataset[T](data: RDD[T])(implicit evidence$5: Encoder[T])
  createDataset[T](data: util.List[T])(implicit evidence$6: Encoder[T])
Press Ctrl+ to choose the selected (or first) suggestion and insert a dot afterwards >>
```

(1) 从 RDD[Case class 类]创建 Dataset

```
val rdd: RDD[Person] = spark.sparkContext.parallelize(Seq(
  Person(1, "zs"),
  Person(2, "ls")
))

import spark.implicits._

// case class 类型的 rdd, 转 dataset
val ds: Dataset[Person] = spark.createDataset(rdd)
val ds2: Dataset[Person] = rdd.toDS()
ds.printSchema()
ds.show()
```

(2) 从 RDD[JavaBean] 创建 Dataset

```
/**
 * 创建一个 javaBean 的 RDD
 * 隐式转换中没有支持好对 javabean 的 encoder 机制
 * 需要自己传入一个 encoder
 * 可以构造一个简单的 encoder，具备序列化功能，但是不具备字段解构的功能
 * 但是，至少能够把一个 RDD[javabean] 变成一个 dataset[javabean]
 * 后续可以通过 rdd 的 map 算子将数据从对象中提取出来，组装成 tuple 元组，然后 toDF 即可进入 sql 空间
 */
val rdd2: RDD[JavaStu] = spark.sparkContext.parallelize(Seq(
  new JavaStu(1, "a", 18, "上海", 99.9),
  new JavaStu(2, "b", 28, "北京", 99.9),
  new JavaStu(3, "c", 38, "西安", 99.9)
))

val encoder = Encoders.kryo(classOf[JavaStu])
val ds2: Dataset[JavaStu] = spark.createDataset(rdd2)(encoder)

val df2: Dataset[Row] = ds2.map(stu => {
  (stu.getId, stu.getName, stu.getAge)
}).toDF("id", "name", "age")
ds2.printSchema()
ds2.show()
df2.show()
```

(3) 从 RDD[其他类] 创建 Dataset

```
/**
 * 将一个 RDD[Map] 变成 Dataset[Map]
 * 2.3.0 版才支持
 *
 */
val rdd3: RDD[Map[String, String]] = spark.sparkContext.parallelize(Seq(
  Map("id" -> "1", "name" -> "zs1"),
  Map("id" -> "2", "name" -> "zs2"),

```

```
Map("id" -> "3", "name" -> "zs3")
))

val ds3: Dataset[Map[String, String]] = rdd3.toDS()
ds3.printSchema()
ds3.show()
```

3.6 RDD/DS/DF 互转

RDD、DataFrame、Dataset 三者有许多共性，有各自适用的场景常常需要在三者之间转换

DataFrame/Dataset 转 RDD:

```
val rdd1: RDD[Row] = testDF.rdd
val rdd2: RDD[T] = testDS.rdd
```

RDD 转 DataFrame:

```
import spark.implicits._
val testDF = rdd.map {line=>
  (line._1, line._2)
}.toDF("col1", "col2")
```

一般用元组把一行的数据写在一起，然后在 toDF 中指定字段名

RDD 转 Dataset:

```
import spark.implicits._
case class Person(col1: String, col2: Int) extends Serializable // 定义字段名和类型
val testDS: Dataset[Person] = rdd.map {line=>
  Person(line._1, line._2)
}.toDS
```

可以注意到，定义每一行的类型 (case class) 时，已经给出了字段名和类型，后面只要往 case class 里面添加值即可

Dataset 转 DataFrame:

这个也很简单，因为只是把 case class 封装成 Row

```
import spark.implicits._  
val testDF:Dataset[Row] = testDS.toDF
```

DataFrame 转 Dataset:

```
import spark.implicits._  
case class Coltest(col1:String,col2:Int)extends Serializable //定义字段名和类型  
val testDS = testDF.as[Coltest]
```

这种方法就是在给出每一列的类型后，使用 as 方法，转成 Dataset，这在数据类型是 DataFrame 又需要针对各个字段处理时极为方便。

在使用一些特殊的操作时，一定要加上 `import spark.implicits._` 不然 toDF、toDS 无法使用

第 4 章 用户自定义函数

通过 spark.udf 功能用户可以自定义函数。

4.1 用户自定义 UDF 函数

```
scala> val df = spark.read.json("examples/src/main/resources/people.json")  
df: org.apache.spark.sql.DataFrame = [age: bigint, name: string]  
  
scala> df.show()  
+----+-----+  
| age |  name |  
+----+-----+  
| null|Michael|  
|  30 |   Andy |  
|  19 |  Justin |
```

```
+-----+
scala> spark.udf.register("addName", (x:String)=> "Name:"+x)
res5: org.apache.spark.sql.expressions.UserDefinedFunction =
UserDefinedFunction(<function1>,StringType,Some(List(StringType)))

scala> df.createOrReplaceTempView("people")

scala> spark.sql("Select addName(name), age from people").show()
+-----+
|UDF:addName(name)| age|
+-----+
|      Name:Michael| null|
|      Name:Andy   |  30|
|      Name:Justin |  19|
+-----+
```

UDF 案例 2

需求，有如下数据

```
id,name,age,height,weight,yanzhi,score
1,a,18,172,120,98,68.8
2,b,28,175,120,97,68.8
3,c,30,180,130,94,88.8
4,d,18,168,110,98,68.8
5,e,26,165,120,98,68.8
6,f,27,182,135,95,89.8
7,g,19,171,122,99,68.8
```

需要计算每一个和其他人之间的余弦相似度（特征向量之间的余弦相似度）

最小化

$$\cos(\theta) = \frac{\sum_{i=1}^n (x_i \times y_i)}{\sqrt{\sum_{i=1}^n (x_i)^2} \times \sqrt{\sum_{i=1}^n (y_i)^2}}$$

$$= \frac{a \bullet b}{||a|| \times ||b||}$$

公式(4)

代码实现:

```
package cn.doitedu.sparksql.udf

import cn.doitedu.sparksql.dataframe.SparkUtil
import org.apache.spark.sql.Row
import org.apache.spark.sql.expressions.UserDefinedFunction

import scala.collection.mutable

/**
 * @date: 2019/10/12
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description:
 * UDF 案例 2 : 用一个自定义函数实现两个向量之间的余弦相似度计算
 */

case class Human(id: Int, name: String, features: Array[Double])

object CosinSimilarity {

  def main(args: Array[String]): Unit = {

    val spark = SparkUtil.getSpark()
    import spark.implicits._
    import spark.sql
```

```
// 加载用户特征数据
val df = spark.read.option("inferSchema", true).option("header",
true).csv("data/features.csv")
df.show()

/**
 * +---+---+---+---+---+---+---+
 * | id|name| age|height|weight|yanzhi|score|
 * +---+---+---+---+---+---+---+
 * | 1| a|18.0| 172.0| 120.0| 98.0| 68.8|
 * | 2| b|28.0| 175.0| 120.0| 97.0| 68.8|
 * | 3| c|30.0| 180.0| 130.0| 94.0| 88.8|
 * | 4| d|18.0| 168.0| 110.0| 98.0| 68.8|
 * | 5| e|26.0| 165.0| 120.0| 98.0| 68.8|
 * | 6| f|27.0| 182.0| 135.0| 95.0| 89.8|
 * | 7| g|19.0| 171.0| 122.0| 99.0| 68.8|
 * +---+---+---+---+---+---+---+
 */

// id,name,age,height,weight,yanzhi,score
// 将用户特征数据组成一个向量(数组)
// 方式 1:
df.rdd.map(row => {
  val id = row.getAs[Int]("id")
  val name = row.getAs[String]("name")
  val age = row.getAs[Double]("age")
  val height = row.getAs[Double]("height")
  val weight = row.getAs[Double]("weight")
  val yanzhi = row.getAs[Double]("yanzhi")
  val score = row.getAs[Double]("score")

  (id, name, Array(age, height, weight, yanzhi, score))
}).toDF("id", "name", "features")

/**
 * +---+---+---+---+
 * | id|name|          features|
 * +---+---+---+---+
 * | 1| a|[18.0, 172.0, 120...|
 * | 2| b|[28.0, 175.0, 120...|
 * | 3| c|[30.0, 180.0, 130...|
 */
```



```
// val cosinSim = (f1:Array[Double],f2:Array[Double])=>{ /* 余弦相似度 */ }
// 开根号的 api:  Math.pow(4.0,0.5)
val cosinSim = (f1:mutable.WrappedArray[Double], f2:mutable.WrappedArray[Double])=>{

    val fenmu1 = Math.pow(f1.map(Math.pow(_,2)).sum,0.5)
    val fenmu2 = Math.pow(f2.map(Math.pow(_,2)).sum,0.5)

    val fenzi = f1.zip(f2).map(tp=>tp._1*tp._2).sum

    fenzi/(fenmu1*fenmu2)
}

// 注册到 sql 引擎:  spark.udf.register("cosin_sim",cosinSim)
spark.udf.register("cos_sim",cosinSim)
joined.createTempView("temp")

// 然后在这个表上计算两人之间的余弦相似度
sql("select id,bid,cos_sim(features,bfeatures) as cos_similary from temp").show()

// 可以自定义函数简单包装一下，就成为一个能生成 column 结果的 dsl 风格函数了
val cossim2: UserDefinedFunction = udf(cosinSim)
joined.select('id,'bid,co$sim2('features,'bfeatures) as "cos_sim").show()

spark.close()
}
}
```

4.2 用户自定义聚合函数 UDAF

弱类型的 `DataFrame` 和强类型的 `Dataset` 都提供了相关的聚合函数，如 `count()`，`countDistinct()`，`avg()`，`max()`，`min()`。

除此之外，用户可以设定自己的自定义 UDAF 聚合函数。

UDAF 的编程模板：

```
/**
 * @date: 2019/10/12
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description:
 *   用户自定义 UDAF 入门示例：求薪资的平均值
 */
object MyAvgUDAF extends UserDefinedAggregateFunction{

  // 函数输入的字段 schema（字段名-字段类型）
  override def inputSchema: StructType = ???

  // 聚合过程中，用于存储局部聚合结果的 schema
  // 比如求平均薪资，中间缓存(局部数据薪资总和,局部数据人数总和)
  override def bufferSchema: StructType = ???

  // 函数的最终返回结果数据类型
  override def dataType: DataType = ???

  // 你这个函数是否是稳定一致的？（对一组相同的输入，永远返回相同的结果），只要是确定的，就写 true
  override def deterministic: Boolean = true

  // 对局部聚合缓存的初始化方法
  override def initialize(buffer: MutableAggregationBuffer): Unit = ???

  // 聚合逻辑所在方法，框架会不断地传入一个新的输入 row，来更新你的聚合缓存数据
  override def update(buffer: MutableAggregationBuffer, input: Row): Unit = ???

  // 全局聚合：将多个局部缓存中的数据，聚合成一个缓存
  // 比如：薪资和薪资累加，人数和人数累加
  override def merge(buffer1: MutableAggregationBuffer, buffer2: Row): Unit = ???

  // 最终输出
  // 比如：从全局缓存中取薪资总和/人数总和
```

```
override def evaluate(buffer: Row): Any = ???  
}
```

核心要义:

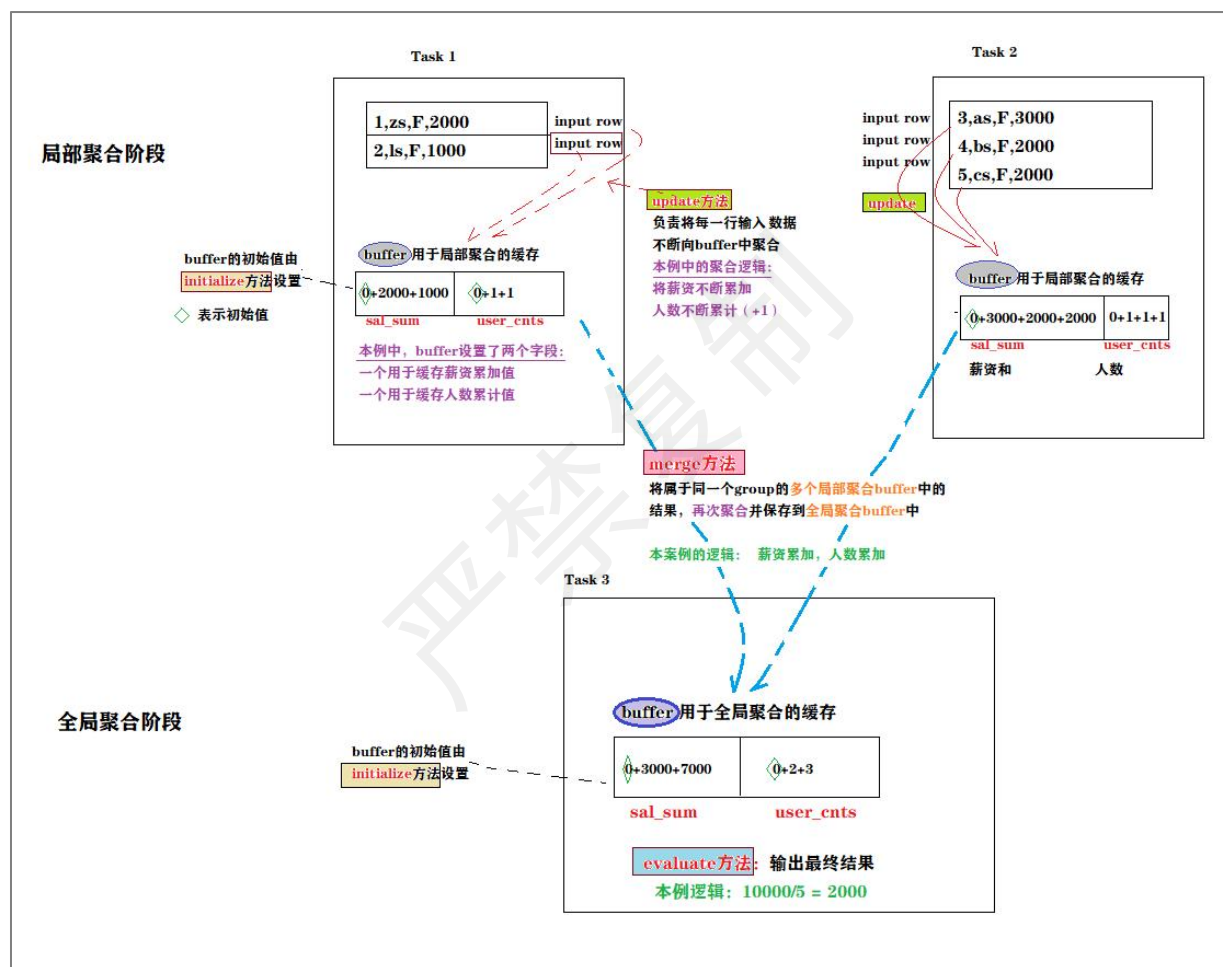
聚合是分步骤进行: 先局部聚合, 再全局聚合

局部聚合 (update) 的结果是保存在一个局部 buffer 中的

全局聚合 (merge) 就是将多个局部 buffer 再聚合成一个 buffer

最后通过 evaluate 将全局聚合的 buffer 中的数据做一个运算得出你要的结果

如下图所示:



4.2.1 弱类型用户自定义聚合函数 UDAF

(1) 需求说明

示例数据:

id	name	sales	discount	state	saleDate
1	Widget Co	1000.0	0.0	AZ	2014-01-01
2	Acme Widgets	2000.0	500.0	CA	2014-02-01
3	Widgetry	1000.0	200.0	CA	2015-01-11
4	Widgets R Us	2000.0	0.0	CA	2015-02-19
5	Ye Olde Widge	3000.0	0.0	MA	2015-02-28

需求: 计算 x 年份的同比上一年份的总销售增长率; 比如 2015 vs 2014 的同比增长

显然, 没有任何一个内置聚合函数可以完成上述需求;

可以多写一些 sql 逻辑来实现, 但如果能自定义一个聚合函数, 当然更方便高效!

Select yearOnyear(saleDate,sales) from t

(2) 自定义 UDAF 实现销售额同比计算

通过继承 UserDefinedAggregateFunction 来实现用户自定义聚合函数。

自定义 UDAF 的代码骨架如下:

```
class UdfMy extends UserDefinedAggregateFunction{
  override def inputSchema: StructType = ???

  override def bufferSchema: StructType = ???

  override def dataType: DataType = ???

  override def deterministic: Boolean = ???

  override def initialize(buffer: MutableAggregationBuffer): Unit = ???

  override def update(buffer: MutableAggregationBuffer, input: Row): Unit = ???
```

```
override def merge(buffer1: MutableAggregationBuffer, buffer2: Row): Unit = ???

override def evaluate(buffer: Row): Any = ???
}
```

完整实现代码如下：

```
/**
 * 工具类
 * @param startDate
 * @param endDate
 */
case class DateRange(startDate: Timestamp, endDate: Timestamp) {
  def contain(targetDate: Date): Boolean = {
    targetDate.before(endDate) && targetDate.after(startDate)
  }
}

/**
 * @date: 2019/10/10
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description: 自定义 UDAF 实现年份销售额同比增长计算
 */
class YearOnYearBasis(current: DateRange) extends UserDefinedAggregateFunction{

  // 聚合函数输入参数的数据类型
  override def inputSchema: StructType = {
    StructType(StructField("metric", DoubleType) :: StructField("timeCategory", DateType) :: Nil)
  }

  // 聚合缓冲区中值得数据类型
  override def bufferSchema: StructType = {
    StructType(StructField("sumOfCurrent", DoubleType) :: StructField("sumOfPrevious",
DoubleType) :: Nil)
  }

  // 返回值的数据类型
  override def dataType: DataType = DoubleType
}
```

```
// 对于相同的输入是否一直返回相同的输出。
override def deterministic: Boolean = true

// 初始化
override def initialize(buffer: MutableAggregationBuffer): Unit = {
  buffer.update(0, 0.0)
  buffer.update(1, 0.0)
}

// 相同 Execute 间的数据合并。
override def update(buffer: MutableAggregationBuffer, input: Row): Unit = {
  if (current.contain(input.getAs[Date](1))) {
    buffer(0) = buffer.getAs[Double](0) + input.getAs[Double](0)
  }
  val previous = DateRange(subtractOneYear(current.startDate),
subtractOneYear(current.endDate))
  if (previous.contain(input.getAs[Date](1))) {
    buffer(1) = buffer.getAs[Double](0) + input.getAs[Double](0)
  }
}

// 不同 Execute 间的数据合并
override def merge(buffer1: MutableAggregationBuffer, buffer2: Row): Unit = {
  buffer1(0) = buffer1.getAs[Double](0) + buffer2.getAs[Double](0)
  buffer1(1) = buffer1.getAs[Double](1) + buffer2.getAs[Double](1)
}

// 计算最终结果
override def evaluate(buffer: Row): Any = {
  if (buffer.getDouble(1) == 0.0)
    0.0
  else
    (buffer.getDouble(0) - buffer.getDouble(1)) / buffer.getDouble(1) * 100
}

def subtractOneYear(d: Timestamp): Timestamp = {
  Timestamp.valueOf(d.toLocalDateTime.minusYears(1))
}
}
```

(3) 补充示例：自定义 UDAF 实现平均薪资计算

下面展示一个求平均工资的自定义聚合函数。

```
package cn.doitedu.sparksql.udf

import org.apache.spark.sql.Row
import org.apache.spark.sql.expressions.{MutableAggregationBuffer,
UserDefinedAggregateFunction}
import org.apache.spark.sql.types.{DataType, DataTypes, StructField, StructType}

/**
 * @date: 2019/10/12
 * @site: www.doitedu.cn
 * @author: hunter.d 涛哥
 * @qq: 657270652
 * @description:
 * 用户自定义 UDAF 入门示例：求薪资的平均值
 */
object MyAvgUDAF extends UserDefinedAggregateFunction {

  // 函数输入的字段 schema（字段名-字段类型）
  override def inputSchema: StructType = StructType(Seq(StructField("salary",
DataTypes.DoubleType)))

  // 聚合过程中，用于存储局部聚合结果的 schema
  // 比如求平均薪资，中间缓存(局部数据薪资总和,局部数据人数总和)
  override def bufferSchema: StructType = StructType(Seq(
    StructField("sum", DataTypes.DoubleType),
    StructField("cnts", DataTypes.LongType)
  ))

  // 函数的最终返回结果数据类型
  override def dataType: DataType = DataTypes.DoubleType

  // 你这个函数是否是稳定一致的？（对一组相同的输入，永远返回相同的结果），只要是确定的，就写 true
  override def deterministic: Boolean = true

  // 对局部聚合缓存的初始化方法
  override def initialize(buffer: MutableAggregationBuffer): Unit = {
    buffer.update(0, 0.0)
  }
}
```

```
buffer.update(1, 0L)
}

// 聚合逻辑所在方法，框架会不断地传入一个新的输入 row，来更新你的聚合缓存数据
override def update(buffer: MutableAggregationBuffer, input: Row): Unit = {

    // 从输入中获取那个人的薪资，加到 buffer 的第一个字段上
    buffer.update(0, buffer.getDouble(0) + input.getDouble(0))

    // 给 buffer 的第 2 个字段加 1
    buffer.update(1, buffer.getLong(1) + 1)

}

// 全局聚合：将多个局部缓存中的数据，聚合成一个缓存
// 比如：薪资和薪资累加，人数和人数累加
override def merge(buffer1: MutableAggregationBuffer, buffer2: Row): Unit = {

    // 把两个 buffer 的字段 1（薪资和）累加到一起，并更新回 buffer1
    buffer1.update(0, buffer1.getDouble(0) + buffer2.getDouble(0))

    // 更新人数
    buffer1.update(1, buffer1.getLong(1) + buffer2.getLong(1))

}

// 最终输出
// 比如：从全局缓存中取薪资总和/人数总和
override def evaluate(buffer: Row): Any = {

    if (buffer.getLong(1) != 0)
        buffer.getDouble(0) / buffer.getLong(1)
    else
        0.0

}
}
```

4.2.2 强类型用户自定义聚合函数

通过继承 `Aggregator` 来实现强类型自定义聚合函数，同样是求平均工资

```
import org.apache.spark.sql.expressions.Aggregator
import org.apache.spark.sql.Encoder
import org.apache.spark.sql.Encoders
import org.apache.spark.sql.SparkSession

// 既然是强类型，可能有 case 类
case class Employee(name: String, salary: Long)
case class Average(var sum: Long, var count: Long)

object MyAverage extends Aggregator[Employee, Average, Double] {
  // 定义一个数据结构，保存工资总数和工资总个数，初始都为 0
  def zero: Average = Average(0L, 0L)
  // Combine two values to produce a new value. For performance, the function may modify `buffer`
  // and return it instead of constructing a new object
  def reduce(buffer: Average, employee: Employee): Average = {
    buffer.sum += employee.salary
    buffer.count += 1
    buffer
  }
  // 聚合不同 execute 的结果
  def merge(b1: Average, b2: Average): Average = {
    b1.sum += b2.sum
    b1.count += b2.count
    b1
  }
  // 计算输出
  def finish(reduction: Average): Double = reduction.sum.toDouble / reduction.count
  // 设定之间值类型的编码器，要转换成 case 类
  // Encoders.product 是进行 scala 元组和 case 类转换的编码器
  def bufferEncoder: Encoder[Average] = Encoders.product
  // 设定最终输出值的编码器
  def outputEncoder: Encoder[Double] = Encoders.scalaDouble
}

import spark.implicits._

val ds = spark.read.json("examples/src/main/resources/employees.json").as[Employee]
ds.show()

// +-----+-----+
// |  name|salary|
// +-----+-----+
// |Michael|  3000|
// |  Andy|  4500|
// | Justin|  3500|
// |  Berta|  4000|
```

```
// +-----+-----+

// Convert the function to a `TypedColumn` and give it a name
val averageSalary = MyAverage.toColumn.name("average_salary")
val result = ds.select(averageSalary)
result.show()

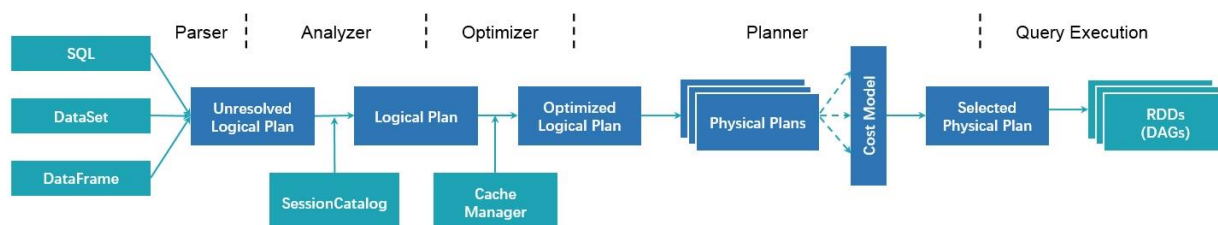
// +-----+
// |average_salary|
// +-----+
// |          3750.0|
// +-----+
```

第 5 章 Spark SQL 的运行原理

正常的 SQL 执行先会经过 SQL Parser 解析 SQL，然后经过 Catalyst 优化器处理，最后到 Spark 执行。而 Catalyst 的过程又分为很多个过程，其中包括：

- Analysis: 主要利用 Catalog 信息将 Unresolved Logical Plan 解析成 Analyzed logical plan;
- Logical Optimizations: 利用一些 Rule (规则) 将 Analyzed logical plan 解析成 Optimized Logical Plan;
- Physical Planning: 前面的 logical plan 不能被 Spark 执行，而这个过程是把 logical plan 转换成多个 physical plans，然后利用代价模型 (cost model) 选择最佳的 physical plan;
- Code Generation: 这个过程会把 SQL 逻辑生成 Java 字节码。

所以整个 SQL 的执行过程可以使用下图表示：



其中蓝色部分就是 Catalyst 优化器处理的部分，也是本章主要讲解的内容。

5.1 元数据管理 SessionCatalog

SessionCatalog 主要用于各种函数资源信息和元数据信息（数据库、数据表、数据视图、数据分区与函数等）的统一管理。

创建临时表或者视图，其实是往 SessionCatalog 注册；

Analyzer 在进行逻辑计划元数据绑定时，也是从 catalog 中获取元数据；

5.2 SQL 解析成逻辑执行计划

当调用 SparkSession 的 sql 或者 SQLContext 的 sql 方法，就会使用 SparkSqlParser 进行 SQL 解析。

Spark 2.0.0 开始引入了第三方语法解析器工具 ANTLR，对 SQL 进行词法分析并构建语法树。

(Antlr 是一款强大的语法生成器工具，可用于读取、处理、执行和翻译结构化的文本或二进制文件，是当前 Java 语言中使用最为广泛的语法生成器工具，我们常见的大数据 SQL 解析都用到了这个工具，包括 Hive、Cassandra、Phoenix、Pig 以及 presto 等) 目前最新版本的 Spark 使用的是 ANTLR4)

它分为 2 个步骤来生成 Unresolved LogicalPlan:

- 词法分析 (SqlBaseLexer) : Lexical Analysis, 负责将 token 分组为符号类
- 语法分析 (SqlBaseParser) : 构建一棵分析树(parse tree)或者抽象语法树 AST (abstract syntax tree)

```
/**
 * The AstBuilder converts an ANTLR4 ParseTree into a catalyst Expression, LogicalPlan or
 * TableIdentifier.
 */
class AstBuilder(conf: SQLConf) extends SqlBaseBaseVisitor[AnyRef] with Logging {
  import ParserUtils._

  def this() = this(new SQLConf())

  protected def typedVisit[T](ctx: ParseTree): T = {
```

具体来说，Spark 基于 presto 的语法文件定义了 Spark SQL 语法文件 SqlBase.g4

(路径 spark-2.4.3/sql/catalyst/src/main/antlr4/org/apache/spark/sql/catalyst/parser/SqlBase.g4)

这个文件定义了 Spark SQL 支持的 SQL 语法。

master spark / sql / catalyst / src / main / antlr4 / org / apache / spark / sql / catalyst / parser / SqlBase.g4

sarutak [SPARK-36371][SQL] Support raw string literal ... ✓

81 contributors +48

1894 lines (1722 sloc) 50.8 KB

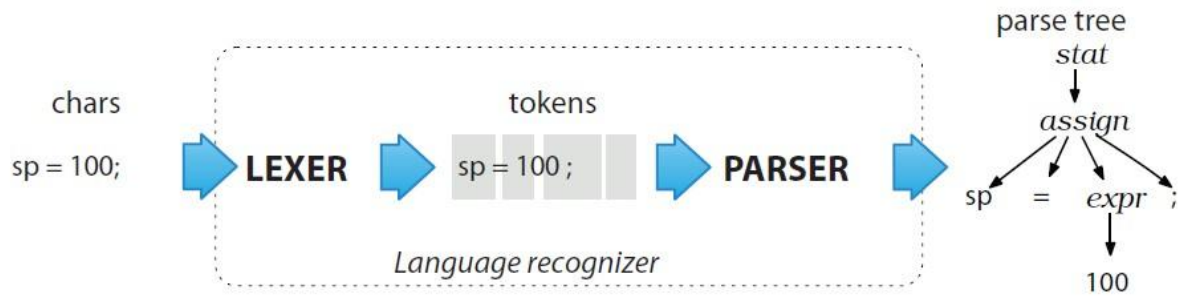
```
1  /*
2   * Licensed under the Apache License, Version 2.0 (the "License");
3   * you may not use this file except in compliance with the License.
4   * You may obtain a copy of the License at
5   *
6   * http://www.apache.org/licenses/LICENSE-2.0
7   *
8   * Unless required by applicable law or agreed to in writing, software
9   * distributed under the License is distributed on an "AS IS" BASIS,
10  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
11  * See the License for the specific language governing permissions and
12  * limitations under the License.
13  *
14  * This file is an adaptation of Presto's presto-parser/src/main/antlr4/com/facebook/presto/sql/parser/SqlBase.g4 grammar.
15  */
16
17 grammar SqlBase;
```

如果我们需要自定义新的语法，需要在这个文件定义好相关语法。然后使用 ANTLR4 对 SqlBase.g4 文件自动解析生成几个 Java 类，其中就包含重要的词法分析器 SqlBaseLexer.java 和语法分析器 SqlBaseParser.java。运行上面的 SQL 会使用 SqlBaseLexer 来解析关键词以及各种标识符等；然后使用 SqlBaseParser 来构建语法树。

下面以一条简单的 SQL 为例进行分析

```
SELECT sum(v)
FROM (
  SELECT
    t1.id,
    1 + 2 + t1.value AS v
  FROM t1 JOIN t2
  WHERE
    t1.id = t2.id AND
    t1.cid = 1 AND
    t1.did = t1.cid + 1 AND
    t2.id > 5) o
```

整个过程就类似于下图。

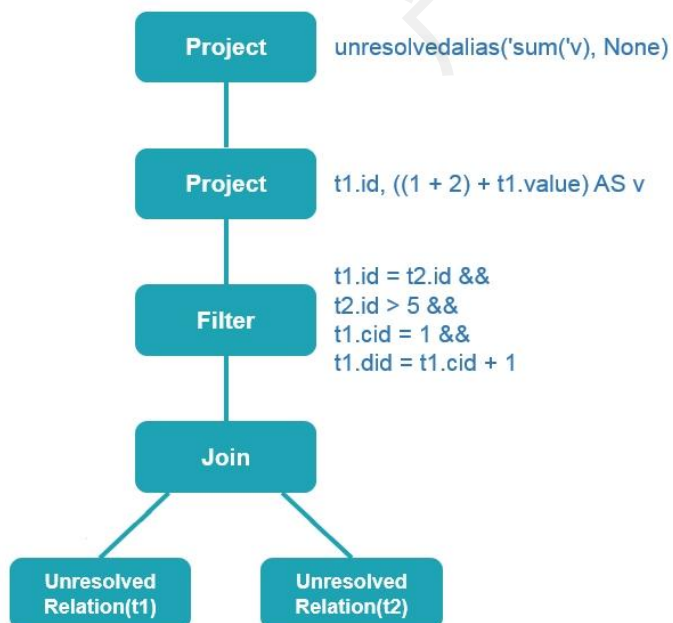


生成语法树之后，使用 AstBuilder 将语法树转换成 LogicalPlan，这个 LogicalPlan 也被称为 Unresolved LogicalPlan。解析后的逻辑计划如下：

```

== Parsed Logical Plan ==
'Project [unresolvedalias('sum('v), None)]
+- 'SubqueryAlias `doitedu_stu`
  +- 'Project ['t1.id, ((1 + 2) + 't1.value) AS v#16]
    +- 'Filter (((('t1.id = 't2.id) && ('t1.cid = 1)) && (('t1.did = ('t1.cid + 1)) && ('t2.id >
5))))
      +- 'Join Inner
        :- 'UnresolvedRelation `t1`
        +- 'UnresolvedRelation `t2`
  
```

图片表示如下：



Unresolved LogicalPlan 是从下往上看，t1 和 t2 两张表被生成了 UnresolvedRelation，过滤的条件、选择的列以及聚合字段都知道了。

Unresolved LogicalPlan 仅仅是一种数据结构，不包含任何数据信息，比如不知道数据源、数据类型，不同的列来自于哪张表等。

5.3 Analyzer 绑定逻辑计划

Analyzer 阶段会使用事先定义好的 Rule 以及 SessionCatalog 等信息对 Unresolved LogicalPlan 进行元数据绑定。

```
/**
 * Provides a logical query plan analyzer, which translates [[UnresolvedAttribute]]s and
 * [[UnresolvedRelation]]s into fully typed objects using information in a [[SessionCatalog]].
 */
class Analyzer(
  catalog: SessionCatalog,
  conf: SQLConf,
  maxIterations: Int)
  extends RuleExecutor[LogicalPlan] with CheckAnalysis {
```

```
class SparkSqlParser(conf: SQLConf) extends AbstractSqlParser(conf) {
  val astBuilder = new SparkSqlAstBuilder(conf)
```

```
  override def parsePlan(sqlText: String): LogicalPlan = parse(sqlText) { parser =>
    astBuilder.visitSingleStatement(parser.singleStatement()) match {
      case plan: LogicalPlan => plan
      case _ =>
        val position = Origin(None, None)
        throw new ParseException(Option(sqlText), "Unsupported SQL statement", position, position)
    }
  }
}
```

Rule 是定义在 Analyzer 里面的，具体如下：

```
lazy val batches: Seq[Batch] = Seq(
  Batch("Hints", fixedPoint,
    new ResolveHints.ResolveBroadcastHints(conf),
    ResolveHints.ResolveCoalesceHints,
```

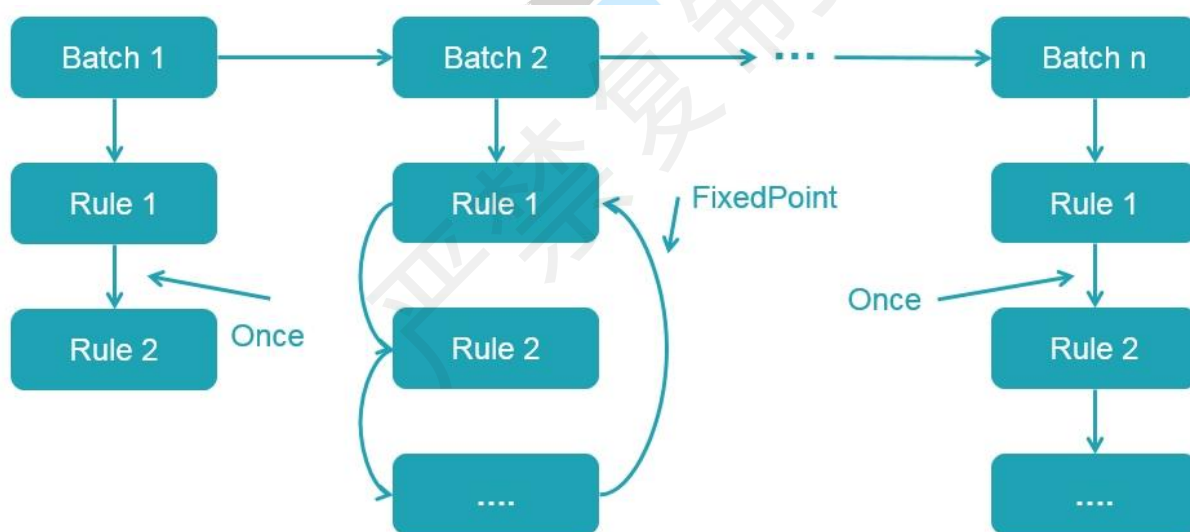
```

    ResolveHints.RemoveAllHints),
Batch("Simple Sanity Check", Once,
    LookupFunctions),
Batch("Substitution", fixedPoint,
    CTESubstitution,
    WindowsSubstitution,
    EliminateUnions,
    new SubstituteUnresolvedOrdinals(conf)),
Batch("Resolution", fixedPoint,
    ResolveTableValuedFunctions ::           //解析表的函数
    ResolveRelations ::                     //解析表或视图
    ResolveReferences ::                   //解析列
    ResolveCreateNamedStruct ::
    ResolveDeserializer ::                 //解析反序列化操作类
    ResolveNewInstance ::
    ResolveUpCast ::                       //解析类型转换
    ResolveGroupingAnalytics ::
    ResolvePivot ::
    ResolveOrdinalInOrderByAndGroupBy ::
    ResolveAggAliasInGroupBy ::
    ResolveMissingReferences ::
    ExtractGenerator ::
    ResolveGenerate ::
    ResolveFunctions ::                     //解析函数
    ResolveAliases ::                       //解析表别名
    ResolveSubquery ::                     //解析子查询
    ResolveSubqueryColumnAliases ::
    ResolveWindowOrder ::
    ResolveWindowFrame ::
    ResolveNaturalAndUsingJoin ::
    ResolveOutputRelation ::
    ExtractWindowExpressions ::
    GlobalAggregates ::
    ResolveAggregateFunctions ::
    TimeWindowing ::
    ResolveInlineTables(conf) ::
    ResolveHigherOrderFunctions(catalog) ::
    ResolveLambdaVariables(conf) ::
    ResolveTimeZone(conf) ::
    ResolveRandomSeed ::
    TypeCoercion.typeCoercionRules(conf) ++
    extendedResolutionRules : _*),
Batch("Post-Hoc Resolution", Once, postHocResolutionRules : _*),

```

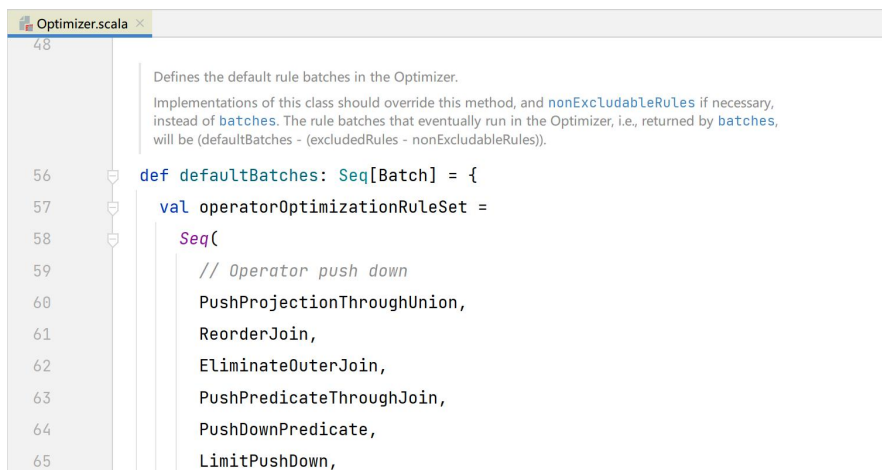
```
Batch("View", Once,
    AliasViewChild(conf)),
Batch("Nondeterministic", Once,
    PullOutNondeterministic),
Batch("UDF", Once,
    HandleNullInputsForUDF),
Batch("FixNullability", Once,
    FixNullability),
Batch("Subquery", Once,
    UpdateOuterReferences),
Batch("Cleanup", fixedPoint,
    CleanupAliases)
)
```

从上面代码可以看出,多个性质类似的 Rule 组成一个 Batch;而多个 Batch 构成一个 batches。这些 batches 会由 RuleExecutor 执行,先按一个一个 Batch 顺序执行,然后对 Batch 里面的每个 Rule 顺序执行。每个 Batch 会执行一次 (Once) 或多次 (FixedPoint, 由 spark.sql.optimizer.maxIterations 参数决定), 执行过程如下:



5.4 Optimizer 优化逻辑计划

优化器也是会定义一套 Rules, 利用这些 Rule 对逻辑计划和 Expression 进行迭代处理, 从而使得树的节点进行合并和优化



```
Optimizer.scala
48
    Defines the default rule batches in the Optimizer.
    Implementations of this class should override this method, and nonExcludableRules if necessary,
    instead of batches. The rule batches that eventually run in the Optimizer, i.e., returned by batches,
    will be (defaultBatches - (excludedRules - nonExcludableRules)).

56 def defaultBatches: Seq[Batch] = {
57   val operatorOptimizationRuleSet =
58     Seq(
59       // Operator push down
60       PushProjectionThroughUnion,
61       ReorderJoin,
62       EliminateOuterJoin,
63       PushPredicateThroughJoin,
64       PushDownPredicate,
65       LimitPushDown,
```

在前文的绑定逻辑计划阶段对 Unresolved LogicalPlan 进行相关 transform 操作得到了 Analyzed Logical Plan, 这个 Analyzed Logical Plan 是可以直接转换成 Physical Plan 然后在 spark 中执行。但是如果直接这么弄的话, 得到的 Physical Plan 很可能不是最优的, 因为在实际应用中, 很多低效的写法会带来执行效率的问题, 需要进一步对 Analyzed Logical Plan 进行处理, 得到更优的逻辑算子树。于是, 针对 SQL 逻辑算子树的优化器 Optimizer 应运而生。

这个阶段的优化器主要是基于规则的 (Rule-based Optimizer, 简称 RBO), 而绝大部分的规则都是启发式规则, 也就是基于直观或经验而得出的规则, 比如列裁剪 (过滤掉查询不需要使用到的列)、谓词下推 (将过滤尽可能地下沉到数据源端)、常量累加 (比如 $1 + 2$ 这种事先计算好) 以及常量替换 (比如 $\text{SELECT } * \text{ FROM table WHERE } i = 5 \text{ AND } j = i + 3$ 可以转换成 $\text{SELECT } * \text{ FROM table WHERE } i = 5 \text{ AND } j = 8$) 等等。

与绑定逻辑计划阶段类似, 这个阶段所有的规则也是实现 Rule 抽象类, 多个规则组成一个 Batch, 多个 Batch 组成一个 batches, 同样也是在 RuleExecutor 中进行执行。

核心源码骨架如下列截图所示:

```

RuleExecutor.scala
67
    Executes the batches of rules defined by the subclass. The batches are executed serially using the
    defined execution strategy. Within each batch, rules are also executed serially.

72 def execute(plan: TreeType): TreeType = {
73     var curPlan = plan
74     val queryExecutionMetrics = RuleExecutor.queryExecutionMeter
75
76     batches.foreach { batch =>
77         val batchStartPlan = curPlan
78         var iteration = 1
79         var lastPlan = curPlan
80         var continue = true

```

```

RuleExecutor.scala Optimizer.scala
    Returns (defaultBatches - (excludedRules - nonExcludableRules)), the rule batches that eventually run in
    the Optimizer.
    Implementations of this class should override defaultBatches, and nonExcludableRules if
    necessary, instead of this method.

244 final override def batches: Seq[Batch] = {
245     val excludedRulesConf =
246         SQLConf.get.optimizerExcludedRules.toSeq.flatMap(Utils.stringToSeq)
247     val excludedRules = excludedRulesConf.filter { ruleName =>
248         val nonExcludable = nonExcludableRules.contains(ruleName)
249         if (nonExcludable) {
250             logWarning(msg = s"Optimization rule '${ruleName}' was not excluded from the optimizer " +
251                 s"because this rule is a non-excludable rule.")
252         }
253         !nonExcludable
254     }
255     if (excludedRules.isEmpty) {
256         defaultBatches
257     } else {
258         defaultBatches.flatMap { batch =>
259             val filteredRules = batch.rules.filter { rule =>

```

```

RuleExecutor.scala Optimizer.scala
    Implementations of this class should override this method, and nonExcludableRules if necessary,
    instead of batches. The rule batches that eventually run in the Optimizer, i.e., returned by batches,
    will be (defaultBatches - (excludedRules - nonExcludableRules)).

56 def defaultBatches: Seq[Batch] = {
57     val operatorOptimizationRuleSet =
58         Seq(
59             // Operator push down
60             PushProjectionThroughUnion,
61             ReorderJoin,
62             EliminateOuterJoin,
63             PushPredicateThroughJoin,
64             PushDownPredicate,
65             LimitPushDown,
66             ColumnPruning,
67             InferFiltersFromConstraints,
68             // Operator combine
69             CollapseRepartition,
70             CollapseProject,
71             CollapseWindow,
72             CombineFilters,
73             CombineLimits,
74             CombineUnions,
75             // Constant folding and strength reduction
76             NullPropagation,
77             ConstantPropagation,
78             FoldablePropagation,

```

The screenshot shows the Spark SQL optimizer code in Scala. The left pane displays the project structure, and the right pane shows the code for the `expressions.scala` file. The code defines two rules: `NullPropagation` and `PushDownPredicate`.

NullPropagation Rule:

```
object NullPropagation extends Rule[LogicalPlan] {
  private def isNullLiteral(e: Expression): Boolean = e match {
    case Literal(null, _) => true
    case _ => false
  }

  def apply(plan: LogicalPlan): LogicalPlan = plan transform {
    case q: LogicalPlan => q transformExpressionsUp {
      case e @ WindowExpression(Cast(Literal(0L, _), _, _, _) =>
        Cast(Literal(0L), e.dataType, Option(SQLConf.get.sessionLocalTimeZone)))
      case e @ AggregateExpression(Count(exprs), _, _, _) if exprs.forall(isNullLiteral) =>
        Cast(Literal(0L), e.dataType, Option(SQLConf.get.sessionLocalTimeZone)))
      case ae @ AggregateExpression(Count(exprs), _, false, _) if !exprs.exists(_.nullable) =>
        // This rule should be only triggered when isDistinct field is false.
        ae.copy(aggregateFunction = Count(Literal(1)))

      case IsNull(c) if !c.nullable => Literal.create(false, BooleanType)
      case IsNotNull(c) if !c.nullable => Literal.create(true, BooleanType)

      case EqualNullSafe(Literal(null, _), r) => IsNull(r)
      case EqualNullSafe(l, Literal(null, _)) => IsNull(l)

      case AssertNotNull(c, _) if !c.nullable => c

      // For Coalesce, remove null literals.
      case e @ Coalesce(children) =>

```

PushDownPredicate Rule:

```
object PushDownPredicate extends Rule[LogicalPlan] with PredicateHelper {
  def apply(plan: LogicalPlan): LogicalPlan = plan transform {
    // SPARK-13473: We can't push the predicate down when the underlying projection output non-
    // deterministic field(s). Non-deterministic expressions are essentially stateful. This
    // implies that, for a given input row, the output are determined by the expression's initial
    // state and all the input rows processed before. In another word, the order of input rows
    // matters for non-deterministic expressions, while pushing down predicates changes the order.
    // This also applies to Aggregate.
    case Filter(condition, project @ Project(fields, grandChild))
      if fields.forall(_.deterministic) && canPushThroughCondition(grandChild, condition) =>

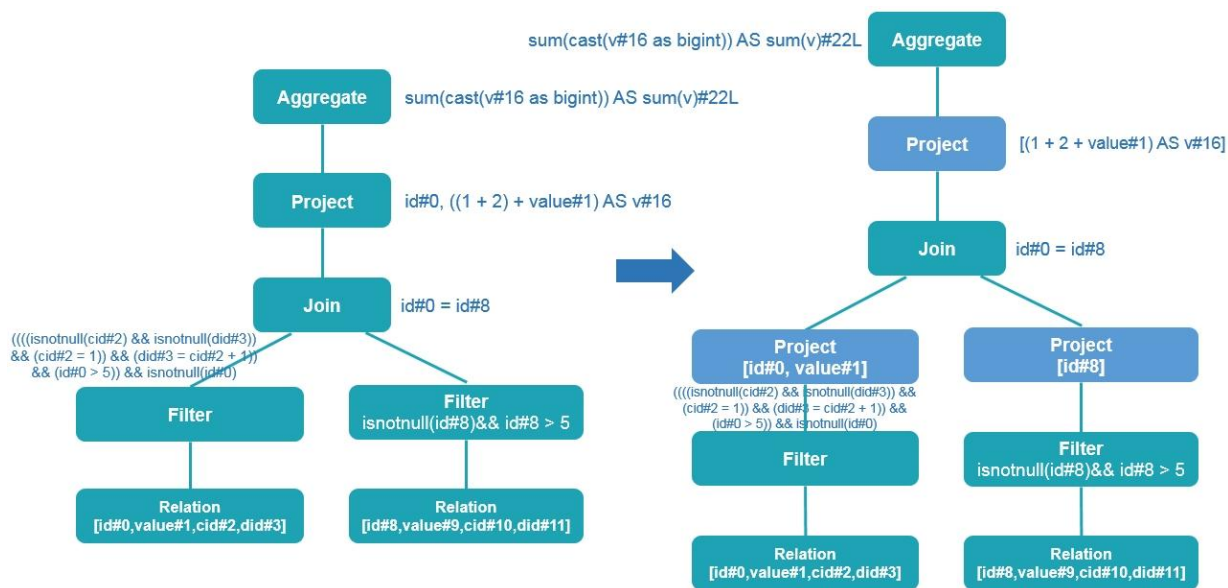
    // Create a map of Aliases to their values from the child projection.
    // e.g., 'SELECT a + b AS c, d ...' produces Map(c -> a + b).
    val aliasMap = AttributeMap(fields.collect {
      case a: Alias => (a.toAttribute, a.child)
    })

    project.copy(child = Filter(replaceAlias(condition, aliasMap), grandChild))
  }
}
```

那么针对前文的 SQL 语句，这个过程都会执行哪些优化呢？下文举例说明。

5.4.1 谓词下推

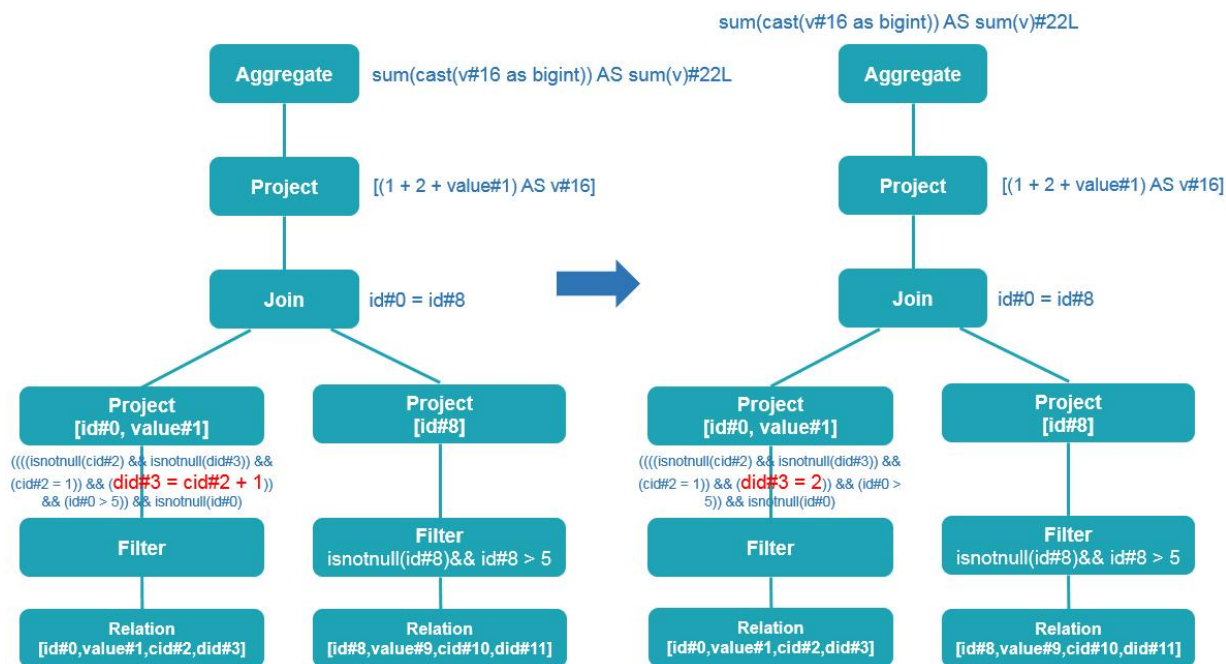
谓词下推在 Spark SQL 是由 `PushDownPredicate` 实现的，这个过程主要将过滤条件尽可能地下推到底层，最好是数据源。上面介绍的 SQL，使用谓词下推优化得到的逻辑计划如下：



从上图可以看出，谓词下推将 Filter 算子直接下推到 Join 之前了（注意，上图是从下往上看）。也就是在扫描 t1 表的时候会先使用 `(((((isnotnull(cid#2) && isnotnull(did#3)) && (cid#2 = 1)) && (did#3 = 2)) && (id#0 > 50000)) && isnotnull(id#0))` 过滤条件过滤出满足条件的数据；同时在扫描 t2 表的时候会先使用 `isnotnull(id#8) && (id#8 > 50000)` 过滤条件过滤出满足条件的数据。经过这样的操作，可以大大减少 Join 算子处理的数据量，从而加快计算速度。

5.4.2 列裁剪

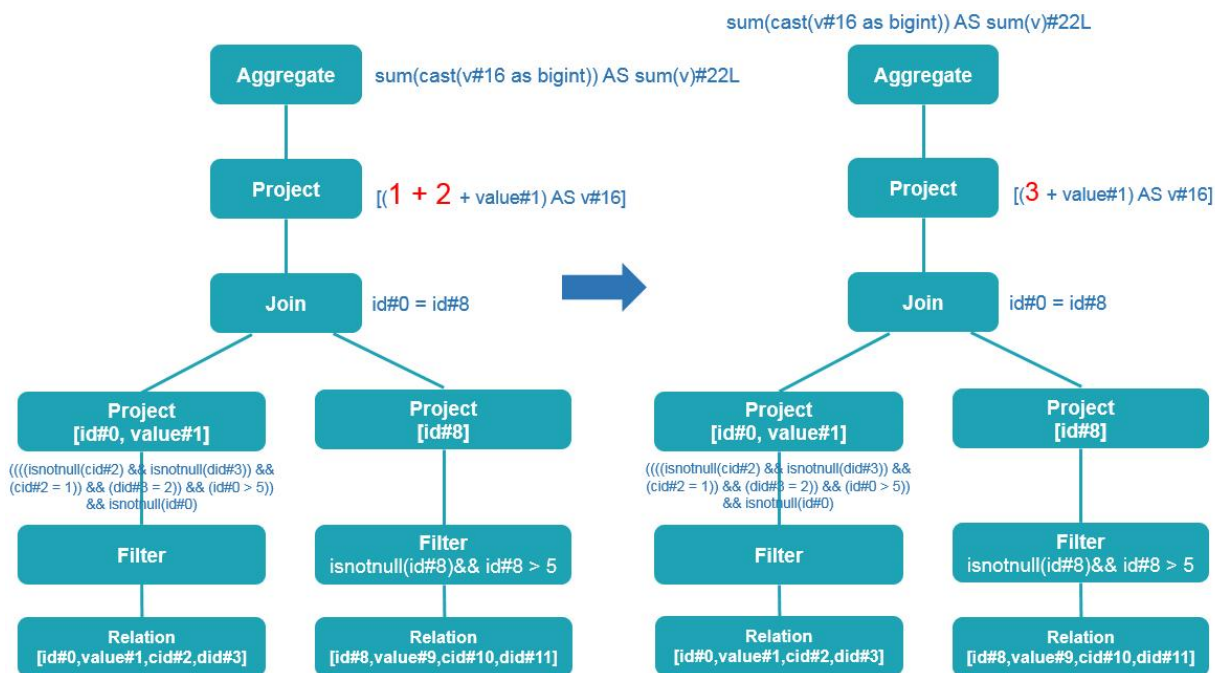
列裁剪在 Spark SQL 是由 ColumnPruning 实现的。因为我们查询的表可能有很多个字段，但是每次查询我们很大可能不需要扫描出所有的字段，这个时候利用列裁剪可以把那些查询不需要的字段过滤掉，使得扫描的数据量减少。所以针对我们上面介绍的 SQL，使用列裁剪优化得到的逻辑计划如下：



从上图可以看出，经过列裁剪后，t1 表只需要查询 id 和 value 两个字段；t2 表只需要查询 id 字段。这样减少了数据的传输，而且如果底层的文件格式为列存（比如 Parquet），可以大大提高数据的扫描速度的。

5.4.3 常量替换

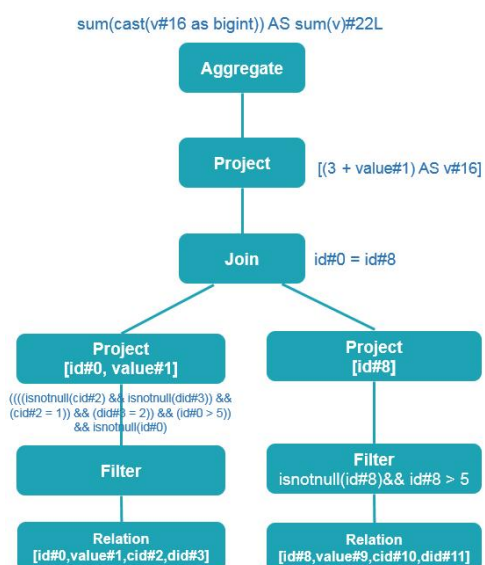
常量替换在 Spark SQL 是由 ConstantPropagation 实现的。也就是将变量替换成常量，比如 `SELECT * FROM table WHERE i = 5 AND j = i + 3` 可以转换成 `SELECT * FROM table WHERE i = 5 AND j = 8`。这个看起来好像没什么的，但是如果扫描的行数非常多可以减少很多的计算时间的开销的。经过这个优化，得到的逻辑计划如下：



我们的查询中有 `t1.cid = 1 AND t1.did = t1.cid + 1` 查询语句，从里面可以看出 `t1.cid` 其实已经是确定的值了，所以我们可以完全使用它计算出 `t1.did`。

5.4.4 常量累加

常量累加在 Spark SQL 是由 ConstantFolding 实现的。这个和常量替换类似，也是在这个阶段把一些常量表达式事先计算好。这个看起来改动的不大，但是在数据量非常大的时候可以减少大量的计算，减少 CPU 等资源的使用。经过这个优化，得到的逻辑计划如下：

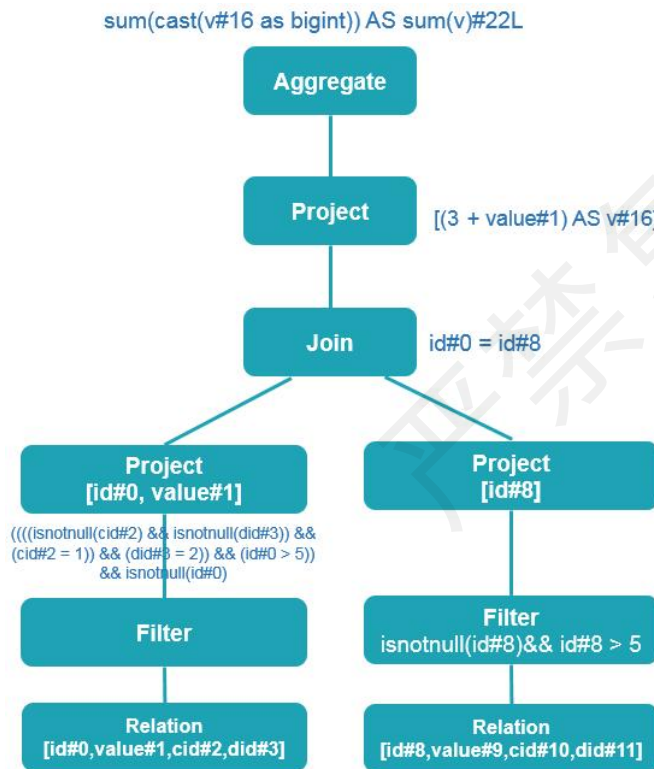


经过上面四个步骤的优化之后，得到的优化之后的逻辑计划为：

```

== Optimized Logical Plan ==
Aggregate [sum(cast(v#16 as bigint)) AS sum(v)#22L]
+- Project [(3 + value#1) AS v#16]
   +- Join Inner, (id#0 = id#8)
      :- Project [id#0, value#1]
         : +- Filter (((((isnotnull(cid#2) && isnotnull(did#3)) && (cid#2 = 1)) && (did#3 = 2)) && (id#0 > 5))
            && isnotnull(id#0))
            : +- Relation[id#0,value#1,cid#2,did#3] csv
      +- Project [id#8]
         +- Filter (isnotnull(id#8) && (id#8 > 5))
            +- Relation[id#8,value#9,cid#10,did#11] csv
  
```

对应的图如下：



到这里，优化逻辑计划阶段就算完成了。另外，Spark 内置提供了多达 70 个优化 Rule，详情请参见

<https://github.com/apache/spark/blob/master/sql/catalyst/src/main/scala/org/apache/spark/sql/catalyst/optimizer/Optimizer.scala#L59>

5.5 使用 SparkPlanner 生成物理计划

SparkSpanner 使用 Planning Strategies，对优化后的逻辑计划进行转换，生成可以执行的物理计划 SparkPlan。

```
/**
 * 将逻辑计划转成物理计划的抽象类。
 * 各实现类通过各种 GenericStrategy 来生成各种可行的待选物理计划。
 * 如一个策略无法对逻辑计划树的所有操作转换，则会调用[GenericStrategy#planLater planLater]]，来获得 一个
 * “占位符”对象暂时填充；之后由[[collectPlaceholders collected]]收集并使用其他策略进行转换
 *
 * TODO: 目前为止，永远只生成一个物理计划
 * 后续迭代中会对“多计划”予以实现
 */
abstract class QueryPlanner[PhysicalPlan <: TreeNode[PhysicalPlan]] {
  /** A list of execution strategies that can be used by the planner */
  def strategies: Seq[GenericStrategy[PhysicalPlan]]

  def plan(plan: LogicalPlan): Iterator[PhysicalPlan] = {
    // 显然，此处还有大量工作需要做，可依然...

    // 收集所有可选的物理计划。
    val candidates = strategies.iterator.flatMap(_(plan))
  }
```

```
abstract class SparkStrategies extends QueryPlanner[SparkPlan] {
  self: SparkPlanner =>

  /**
   * Plans special cases of limit operators.
   */
  object SpecialLimits extends Strategy {

class SparkPlanner(
  val sparkContext: SparkContext,
  val conf: SQLConf,
  val experimentalMethods: ExperimentalMethods)
  extends SparkStrategies {
```

逻辑计划翻译成物理计划时，使用的是策略 (Strategy) ；

前面介绍的逻辑计划绑定和优化经过 Transformations 动作之后，树的类型并没有改变，

Logical Plan 转化成物理计划后，树的类型改变了，由 Logical Plan 转换成 Physical Plan 了。

一个逻辑计划 (Logical Plan) 经过一系列的策略处理之后，得到多个物理计划 (Physical Plans)，物理计划在 Spark 是由 SparkPlan 实现的。

多个物理计划经过代价模型 (Cost Model) 得到选择后的物理计划 (Selected Physical Plan)，整个过程如下所示：



Cost Model 对应的就是基于代价的优化 (Cost-based Optimizations, CBO，主要由华为的大佬们实现的，详见 SPARK-16026)，核心思想是计算每个物理计划的代价，然后得到最优的物理计划。目前，这一部分并没有实现，直接返回多个物理计划列表的第一个作为最优的物理计划，如下：

```
lazy val sparkPlan: SparkPlan = {
  SparkSession.setActiveSession(sparkSession)
  // TODO: We use next(), i.e. take the first plan returned by the planner, here for now,
  //       but we will implement to choose the best plan.
  planner.plan(ReturnAnswer(optimizedPlan)).next()
}
```

而 SPARK-16026 引入的 CBO 优化主要是在前面介绍的优化逻辑计划阶段 - Optimizer 阶段进行的，对应的 Rule 为 CostBasedJoinReorder，并且默认是关闭的，需要通过 `spark.sql.cbo.enabled` 或 `spark.sql.cbo.joinReorder.enabled` 参数开启。

所以到了这个节点，最后得到的物理计划如下：

```
== Physical Plan ==
*(3) HashAggregate(keys=[], functions=[sum(cast(v#16 as bigint))], output=[sum(v) #22L])
+- Exchange SinglePartition
   +- *(2) HashAggregate(keys=[], functions=[partial_sum(cast(v#16 as bigint))], output=[sum#24L])
      +- *(2) Project [(3 + value#1) AS v#16]
```

```

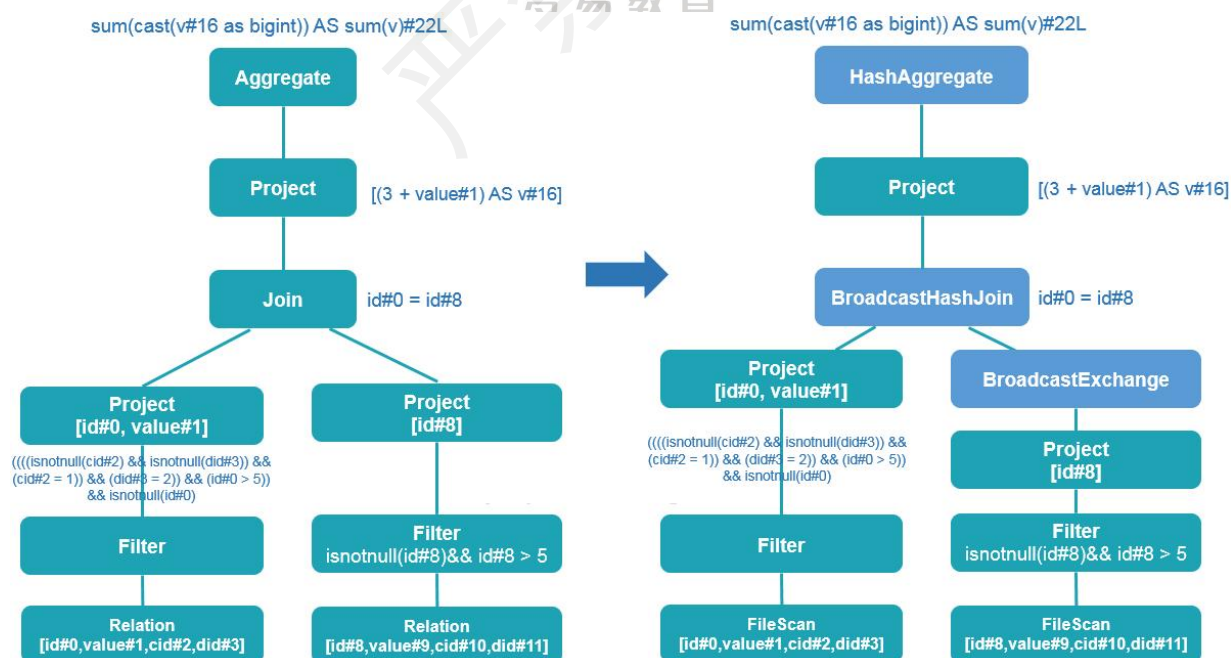
+- *(2) BroadcastHashJoin [id#0], [id#8], Inner, BuildRight
:- *(2) Project [id#0, value#1]
: +- *(2) Filter (((isnotnull(cid#2) && isnotnull(did#3)) && (cid#2 = 1)) && (did#3 = 2)) && (id#0 > 5)) && isnotnull(id#0))
: +- *(2) FileScan csv [id#0,value#1,cid#2,did#3] Batched: false, Format: CSV,
Location: InMemoryFileIndex[file:/iteblog/t1.csv], PartitionFilters: [], PushedFilters:
[IsNotNull(cid), IsNotNull(did), EqualTo(cid,1), EqualTo(did,2), GreaterThan(id,5), IsNotNull(id)],
ReadSchema: struct<id:int,value:int,cid:int,did:int>
+- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as
bigint)))

+- *(1) Project [id#8]
+- *(1) Filter (isnotnull(id#8) && (id#8 > 5))
+- *(1) FileScan csv [id#8] Batched: false, Format: CSV, Location:
InMemoryFileIndex[file:/iteblog/t2.csv], PartitionFilters: [], PushedFilters: [IsNotNull(id),
GreaterThan(id,5)], ReadSchema: struct<id:int>

```

从上面的结果可以看出，物理计划阶段已经知道数据源是从 csv 文件里面读取了，也知道文件的路径，数据类型等。而且在读取文件的时候，直接将过滤条件（PushedFilters）加进去了。

同时，这个 Join 变成了 BroadcastHashJoin，也就是将 t2 表的数据 Broadcast 到 t1 表所在的节点。图表示如下：



到这里，Physical Plan 就完全生成了。

5.6 从物理执行计划获取 inputRdd 执行

从物理计划上，获取 inputRdd

从物理计划上，生成全阶段代码，并编译反射出迭代器 newBiIterator 的Clazz

[真名：BufferedRowIterator]

然后将 inputRDD 做一个 transformation 得到最终要执行的 rdd

```
inputRdd.mapPartitionsWithIndex((index, iter) => {  
    new newBiIterator() {  
        hasNext() {  
            iter.hasNext  
        }  
        next() {  
            processNext(iter.next())  
        }  
    }  
})
```

然后，对最后返回的 rdd，执行你所需要的行动算子

```
rdd.collect().foreach(println)
```

第 6 章 Spark SQL 实战

6.1 数据说明

数据集是货品交易数据集。

tbDate	tbStockDetail	tbStock
dateid: date 日期	ordernumber: varchar 订单号	ordernumber: varchar 订单号
years: varchar 年月	rownum: varchar 行号	locationid: varchar 交易位置
theyear: varchar 年	itemid: varchar 货品	dateid: date 交易日期
month: varchar 月	number: varchar 数量	
day: varchar 日	price: varchar 单价	
weekday: varchar 周几	amount: int 销售额	
week: varchar 第几周		
quarter: varchar 季度		
period: varchar 旬		
halfmonth: varchar 半月		

每个订单可能包含多个货品，每个订单可以产生多次交易，不同的货品有不同的单价。

6.2 加载数据

tbStock:

```
scala> case class tbStock(ordernumber:String,locationid:String,dateid:String) extends
Serializable
defined class tbStock

scala> val tbStockRdd = spark.sparkContext.textFile("tbStock.txt")
tbStockRdd: org.apache.spark.rdd.RDD[String] = tbStock.txt MapPartitionsRDD[1] at textFile at
<console>:23

scala> val tbStockDS =
tbStockRdd.map(_._split(",")).map(attr=>tbStock(attr(0),attr(1),attr(2))).toDS
tbStockDS: org.apache.spark.sql.Dataset[tbStock] = [ordernumber: string, locationid: string ...
1 more field]

scala> tbStockDS.show()
+-----+-----+-----+
| ordernumber|locationid|  dataid|
+-----+-----+-----+
| BYSL00000893|    ZHAO|2007-8-23|
| BYSL00000897|    ZHAO|2007-8-24|
| BYSL00000898|    ZHAO|2007-8-25|
```

```
|BYSL00000899|      ZHAO|2007-8-26|
|BYSL00000900|      ZHAO|2007-8-26|
|BYSL00000901|      ZHAO|2007-8-27|
|BYSL00000902|      ZHAO|2007-8-27|
|BYSL00000904|      ZHAO|2007-8-28|
|BYSL00000905|      ZHAO|2007-8-28|
|BYSL00000906|      ZHAO|2007-8-28|
|BYSL00000907|      ZHAO|2007-8-29|
|BYSL00000908|      ZHAO|2007-8-30|
|BYSL00000909|      ZHAO| 2007-9-1|
|BYSL00000910|      ZHAO| 2007-9-1|
|BYSL00000911|      ZHAO|2007-8-31|
|BYSL00000912|      ZHAO| 2007-9-2|
|BYSL00000913|      ZHAO| 2007-9-3|
|BYSL00000914|      ZHAO| 2007-9-3|
|BYSL00000915|      ZHAO| 2007-9-4|
|BYSL00000916|      ZHAO| 2007-9-4|
```

```
+-----+-----+-----+
only showing top 20 rows
```

tbStockDetail:

```
scala> case class tbStockDetail(ordernumber:String, rownum:Int, itemid:String, number:Int,
price:Double, amount:Double) extends Serializable
defined class tbStockDetail

scala> val tbStockDetailRdd = spark.sparkContext.textFile("tbStockDetail.txt")
tbStockDetailRdd: org.apache.spark.rdd.RDD[String] = tbStockDetail.txt MapPartitionsRDD[13] at
textFile at <console>:23

scala> val tbStockDetailDS = tbStockDetailRdd.map(_._split(",")).map(attr=>
tbStockDetail(attr(0),attr(1).trim().toInt,attr(2),attr(3).trim().toInt,attr(4).trim().toDou
ble, attr(5).trim().toDouble)).toDS
tbStockDetailDS: org.apache.spark.sql.Dataset[tbStockDetail] = [ordernumber: string, rownum:
int ... 4 more fields]

scala> tbStockDetailDS.show()
+-----+-----+-----+-----+-----+-----+
| ordernumber|rownum|      itemid|number|price|amount|
+-----+-----+-----+-----+-----+-----+
|BYSL00000893|    0|FS527258160501|   -1|268.0|-268.0|
```

BYSL00000893	1	FS527258169701	1	268.0	268.0
BYSL00000893	2	FS527230163001	1	198.0	198.0
BYSL00000893	3	24627209125406	1	298.0	298.0
BYSL00000893	4	K9527220210202	1	120.0	120.0
BYSL00000893	5	01527291670102	1	268.0	268.0
BYSL00000893	6	QY527271800242	1	158.0	158.0
BYSL00000893	7	ST040000010000	8	0.0	0.0
BYSL00000897	0	04527200711305	1	198.0	198.0
BYSL00000897	1	MY627234650201	1	120.0	120.0
BYSL00000897	2	01227111791001	1	249.0	249.0
BYSL00000897	3	MY627234610402	1	120.0	120.0
BYSL00000897	4	01527282681202	1	268.0	268.0
BYSL00000897	5	84126182820102	1	158.0	158.0
BYSL00000897	6	K9127105010402	1	239.0	239.0
BYSL00000897	7	QY127175210405	1	199.0	199.0
BYSL00000897	8	24127151630206	1	299.0	299.0
BYSL00000897	9	G1126101350002	1	158.0	158.0
BYSL00000897	10	FS527258160501	1	198.0	198.0
BYSL00000897	11	ST040000010000	13	0.0	0.0

+-----+-----+-----+-----+-----+

only showing top 20 rows

tbDate:

```
scala> case class tbDate(dateid:String, years:Int, theyear:Int, month:Int, day:Int, weekday:Int,
week:Int, quarter:Int, period:Int, halfmonth:Int) extends Serializable
defined class tbDate
```

```
scala> val tbDateRdd = spark.sparkContext.textFile("tbDate.txt")
tbDateRdd: org.apache.spark.rdd.RDD[String] = tbDate.txt MapPartitionsRDD[20] at textFile at
<console>:23
```

```
scala> val tbDateDS = tbDateRdd.map(_._split(",")).map(attr=>
tbDate(attr(0),attr(1).trim().toInt, attr(2).trim().toInt,attr(3).trim().toInt,
attr(4).trim().toInt, attr(5).trim().toInt, attr(6).trim().toInt, attr(7).trim().toInt,
attr(8).trim().toInt, attr(9).trim().toInt)).toDS
tbDateDS: org.apache.spark.sql.Dataset[tbDate] = [dateid: string, years: int ... 8 more fields]
```

```
scala> tbDateDS.show()
```

dateid	years	theyear	month	day	weekday	week	quarter	period	halfmonth
2003-1-1	200301	2003	1	1	3	1	1	1	1
2003-1-2	200301	2003	1	2	4	1	1	1	1
2003-1-3	200301	2003	1	3	5	1	1	1	1
2003-1-4	200301	2003	1	4	6	1	1	1	1
2003-1-5	200301	2003	1	5	7	1	1	1	1
2003-1-6	200301	2003	1	6	1	2	1	1	1
2003-1-7	200301	2003	1	7	2	2	1	1	1
2003-1-8	200301	2003	1	8	3	2	1	1	1
2003-1-9	200301	2003	1	9	4	2	1	1	1
2003-1-10	200301	2003	1	10	5	2	1	1	1
2003-1-11	200301	2003	1	11	6	2	1	2	1
2003-1-12	200301	2003	1	12	7	2	1	2	1
2003-1-13	200301	2003	1	13	1	3	1	2	1
2003-1-14	200301	2003	1	14	2	3	1	2	1
2003-1-15	200301	2003	1	15	3	3	1	2	1
2003-1-16	200301	2003	1	16	4	3	1	2	2
2003-1-17	200301	2003	1	17	5	3	1	2	2
2003-1-18	200301	2003	1	18	6	3	1	2	2
2003-1-19	200301	2003	1	19	7	3	1	2	2
2003-1-20	200301	2003	1	20	1	4	1	2	2

only showing top 20 rows

注册表:

```
scala> tbStockDS.createOrReplaceTempView("tbStock")

scala> tbDateDS.createOrReplaceTempView("tbDate")

scala> tbStockDetailDS.createOrReplaceTempView("tbStockDetail")
```

6.3 计算所有数据中每年的销售单数、销售总额

统计所有订单中每年的销售单数、销售总额

三个表连接后以 count(distinct a.ordernumber)计销售单数，sum(b.amount)计销售总额

tbDate	tbStockDetail	tbStock
dateid: date 日期	ordernumber: varchar 订单号	ordernumber: varchar 订单号
years: varchar 年月	rownum: varchar 行号	locationid: varchar 交易位置
theyear: varchar 年	itemid: varchar 货品	dateid: date 交易日期
month: varchar 月	number: varchar 数量	
day: varchar 日	price: varchar 单价	
weekday: varchar 周几	amount: int 销售额	
week: varchar 第几周		
quarter: varchar 季度		
period: varchar 旬		
halfmonth: varchar 半月		

```
SELECT c.theyear, COUNT(DISTINCT a.ordernumber), SUM(b.amount)
FROM tbStock a
    JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
    JOIN tbDate c ON a.dateid = c.dateid
GROUP BY c.theyear
ORDER BY c.theyear
```

```
spark.sql("SELECT c.theyear, COUNT(DISTINCT a.ordernumber), SUM(b.amount) FROM tbStock a JOIN
tbStockDetail b ON a.ordernumber = b.ordernumber JOIN tbDate c ON a.dateid = c.dateid GROUP BY
c.theyear ORDER BY c.theyear").show
```

结果如下:

theyear	count (DISTINCT ordernumber)	sum(amount)
2004	1094	3268115.4991999999
2005	3828	1.3257564149999999E7
2006	3772	1.3680982900000000E7
2007	4885	1.6719354559999993E7
2008	4861	1.4674295300000001E7
2009	2619	6323697.189999999
2010	94	210949.65999999997

6.4 查询每年最大金额的订单及其金额

目标: 统计每年最大金额订单的销售额:

tbDate	tbStockDetail	tbStock
dateid: date 日期	ordernumber: varchar 订单号	ordernumber: varchar 订单号
years: varchar 年月	rownum: varchar 行号	locationid: varchar 交易位置
theyear: varchar 年	itemid: varchar 货品	dateid: date 交易日期
month: varchar 月	number: varchar 数量	
day: varchar 日	price: varchar 单价	
weekday: varchar 周几	amount: int 销售金额	
week: varchar 第几周		
quarter: varchar 季度		
period: varchar 旬		
halfmonth: varchar 半月		

1. 统计每年，每个订单一共有多少销售额

```
SELECT a.dateid, a.ordernumber, SUM(b.amount) AS SumOfAmount
FROM tbStock a
      JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
GROUP BY a.dateid, a.ordernumber
```

```
spark.sql("SELECT a.dateid, a.ordernumber, SUM(b.amount) AS SumOfAmount FROM tbStock a JOIN
tbStockDetail b ON a.ordernumber = b.ordernumber GROUP BY a.dateid, a.ordernumber").show
```

2. 结果如下:

dateid	ordernumber	SumOfAmount
2008-4-9	BYSL00001175	350.0
2008-5-12	BYSL00001214	592.0
2008-7-29	BYSL00011545	2064.0
2008-9-5	DGSL00012056	1782.0
2008-12-1	DGSL00013189	318.0
2008-12-18	DGSL00013374	963.0
2009-8-9	DGSL00015223	4655.0
2009-10-5	DGSL00015585	3445.0
2010-1-14	DGSL00016374	2934.0
2006-9-24	GCSL00000673	3556.10000000000004
2007-1-26	GCSL00000826	9375.19999999999999
2007-5-24	GCSL00001020	6171.30000000000002
2008-1-8	GCSL00001217	7601.6
2008-9-16	GCSL00012204	2018.0
2006-7-27	GHSL00000603	2835.6
2006-11-15	GHSL00000741	3951.94
2007-6-6	GHSL00001149	0.0
2008-4-18	GHSL00001631	12.0

2008-7-15 GHSL00011367	578.0
2009-5-8 GHSL00014637	1797.6
+-----+	

3. 以上一步查询结果为基础表，和表 tbDate 使用 dateid join，求出每年最大金额订单的销售

```
SELECT theyear, MAX(c.SumOfAmount) AS SumOfAmount
FROM (SELECT a.dateid, a.ordernumber, SUM(b.amount) AS SumOfAmount
      FROM tbStock a
      JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
      GROUP BY a.dateid, a.ordernumber
     ) c
JOIN tbDate d ON c.dateid = d.dateid
GROUP BY theyear
ORDER BY theyear DESC
```

```
spark.sql("SELECT theyear, MAX(c.SumOfAmount) AS SumOfAmount FROM (SELECT a.dateid, a.ordernumber, SUM(b.amount) AS SumOfAmount FROM tbStock a JOIN tbStockDetail b ON a.ordernumber = b.ordernumber GROUP BY a.dateid, a.ordernumber ) c JOIN tbDate d ON c.dateid = d.dateid GROUP BY theyear ORDER BY theyear DESC").show
```

4. 结果如下：

+-----+	
theyear	SumOfAmount
+-----+	
2010	13065.280000000002
2009	25813.200000000008
2008	55828.0
2007	159126.0
2006	36124.0
2005	38186.399999999994
2004	23656.799999999997
+-----+	

6.5 计算每年最畅销货品

目标 1：统计每年最畅销货品（哪个货品销售额 amount 在当年最高，哪个就是最畅销货品）

目标 2：统计每年最畅销货品（哪个货品销售数量当年最高，哪个就是最畅销货品）

tbDate	tbStockDetail	tbStock
dateid: date 日期	ordernumber: varchar 订单号	ordernumber: varchar 订单号
years: varchar 年月	rownum: varchar 行号	locationid: varchar 交易位置
theyear: varchar 年	itemid: varchar 货品	dateid: date 交易日期
month: varchar 月	number: varchar 数量	
day: varchar 日	price: varchar 单价	
weekday: varchar 周几	amount: int 销售金额	
week: varchar 第几周		
quarter: varchar 季度		
period: varchar 旬		
halfmonth: varchar 半月		

第一步、求出每年每个货品的销售金额

```
SELECT c.theyear, b.itemid, SUM(b.amount) AS SumOfAmount
FROM tbStock a
    JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
    JOIN tbDate c ON a.dateid = c.dateid
GROUP BY c.theyear, b.itemid
```

```
spark.sql("SELECT c.theyear, b.itemid, SUM(b.amount) AS SumOfAmount FROM tbStock a JOIN
tbStockDetail b ON a.ordernumber = b.ordernumber JOIN tbDate c ON a.dateid = c.dateid GROUP BY
c.theyear, b.itemid").show
```

结果如下：

theyear	itemid	SumOfAmount
2004	43824480810202	4474.72
2006	YA214325360101	556.0
2006	BT624202120102	360.0
2007	AK215371910101	24603.639999999992
2008	AK216169120201	29144.199999999997
2008	YL526228310106	16073.099999999999
2009	KM529221590106	5124.8000000000001
2004	HT224181030201	2898.6000000000004
2004	SG224308320206	7307.06
2007	04426485470201	14468.800000000001
2007	84326389100102	9134.11
2007	B4426438020201	19884.2
2008	YL427437320101	12331.799999999997
2008	MH215303070101	8827.0
2009	YL629228280106	12698.4
2009	BL529298020602	2415.8
2009	F5127363019006	614.0

	2005 24425428180101	34890.74
	2007 YA214127270101	240.0
	2007 MY127134830105	11099.92
+-----+		

第二步：在第一步的基础上，统计每年单个货品中的最大金额

```
SELECT d.theyear, MAX(d.SumOfAmount) AS MaxOfAmount
FROM (SELECT c.theyear, b.itemid, SUM(b.amount) AS SumOfAmount
      FROM tbStock a
        JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
        JOIN tbDate c ON a.dateid = c.dateid
      GROUP BY c.theyear, b.itemid
    ) d
GROUP BY d.theyear
```

```
spark.sql("SELECT d.theyear, MAX(d.SumOfAmount) AS MaxOfAmount FROM (SELECT c.theyear, b.itemid, SUM(b.amount) AS SumOfAmount FROM tbStock a JOIN tbStockDetail b ON a.ordernumber = b.ordernumber JOIN tbDate c ON a.dateid = c.dateid GROUP BY c.theyear, b.itemid ) d GROUP BY d.theyear").show
```

结果如下：

	theyear	MaxOfAmount
+-----+		
	2007	70225.1
	2006	113720.6
	2004	53401.759999999995
	2009	30029.2
	2005	56627.329999999994
	2010	4494.0
	2008	98003.600000000003
+-----+		

第三步：用最大销售额和统计好的每个货品的销售额 join，以及用年 join，集合得到最畅销货品那一行信息

```
SELECT DISTINCT e.theyear, e.itemid, f.MaxOfAmount
FROM (SELECT c.theyear, b.itemid, SUM(b.amount) AS SumOfAmount
      FROM tbStock a
        JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
        JOIN tbDate c ON a.dateid = c.dateid
      GROUP BY c.theyear, b.itemid
    ) e
JOIN (SELECT d.theyear, MAX(d.SumOfAmount) AS MaxOfAmount
      FROM (SELECT c.theyear, b.itemid, SUM(b.amount) AS SumOfAmount
```

```
FROM tbStock a
    JOIN tbStockDetail b ON a.ordernumber = b.ordernumber
    JOIN tbDate c ON a.dateid = c.dateid
GROUP BY c.theyear, b.itemid
) d
GROUP BY d.theyear
) f ON e.theyear = f.theyear
AND e.SumOfAmount = f.MaxOfAmount
ORDER BY e.theyear
```

```
spark.sql("SELECT DISTINCT e.theyear, e.itemid, f.maxofamount FROM (SELECT c.theyear, b.itemid, SUM(b.amount) AS sumofamount FROM tbStock a JOIN tbStockDetail b ON a.ordernumber = b.ordernumber JOIN tbDate c ON a.dateid = c.dateid GROUP BY c.theyear, b.itemid ) e JOIN (SELECT d.theyear, MAX(d.sumofamount) AS maxofamount FROM (SELECT c.theyear, b.itemid, SUM(b.amount) AS sumofamount FROM tbStock a JOIN tbStockDetail b ON a.ordernumber = b.ordernumber JOIN tbDate c ON a.dateid = c.dateid GROUP BY c.theyear, b.itemid ) d GROUP BY d.theyear ) f ON e.theyear = f.theyear AND e.sumofamount = f.maxofamount ORDER BY e.theyear").show
```

结果如下：

theyear	itemid	maxofamount
2004	JY424420810101	53401.759999999995
2005	24124118880102	56627.329999999994
2006	JY425468460101	113720.6
2007	JY425468460101	70225.1
2008	E2628204040101	98003.600000000003
2009	YL327439080102	30029.2
2010	SQ429425090101	4494.0

第 7 章 JDBC/ODBC 服务器

Spark SQL 也提供 JDBC 连接支持,这对于让商业智能(BI)工具连接到 Spark 集群上以及在多用户间共享一个集群的场景都非常有用。JDBC 服务器作为一个独立的 Spark 驱动器程序运行,可以在多用户之间共享。任意一个客户端都可以在内存中缓存数据表,对表进行查询。集群的资源以及缓存数据都在所有用户之间共享。

Spark SQL 的 JDBC 服务器与 Hive 中的 HiveServer2 相一致。由于使用了 Thrift 通信协议,它也

被称为“Thrift server”。

服务器可以通过 Spark 目录中的 `sbin/start-thriftserver.sh` 启动。这个脚本接受的参数选项大多与 `spark-submit` 相同。默认情况下，服务器会在 `localhost:10000` 上进行监听，我们可以通过环境变量(`HIVE_SERVER2_THRIFT_PORT` 和 `HIVE_SERVER2_THRIFT_BIND_HOST`)修改这些设置，也可以通过 Hive 配置选项(`hive.server2.thrift.port` 和 `hive.server2.thrift.bind.host`)来修改。

你也可以通过命令行参数：`--hiveconf property=value` 来设置 Hive 选项。

```
./sbin/start-thriftserver.sh \
--hiveconf hive.server2.thrift.port=<listening-port> \
--hiveconf hive.server2.thrift.bind.host=<listening-host> \
--master <master-uri>
...
./bin/beeline
beeline> !connect jdbc:hive2://master01:10000
```

在 Beeline 客户端中，你可以使用标准的 HiveQL 命令来创建、列举以及查询数据表。

```
[bigdata@master01 spark-2.1.1-bin-hadoop2.7]$ ./sbin/start-thriftserver.sh
starting      org.apache.spark.sql.hive.thriftserver.HiveThriftServer2,      logging      to
/home/bigdata/hadoop/spark-2.1.1-bin-hadoop2.7/logs/spark-bigdata-org.apache.spark.sql.hive.th
riftserver.HiveThriftServer2-1-master01.out
[bigdata@master01 spark-2.1.1-bin-hadoop2.7]$ ./bin/beeline
Beeline version 1.2.1.spark2 by Apache Hive
beeline> !connect jdbc:hive2://master01:10000
Connecting to jdbc:hive2://master01:10000
Enter username for jdbc:hive2://master01:10000: bigdata
Enter password for jdbc:hive2://master01:10000: *****
log4j:WARN No appenders could be found for logger (org.apache.hive.jdbc.Utills).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Connected to: Spark SQL (version 2.1.1)
Driver: Hive JDBC (version 1.2.1.spark2)
Transaction isolation: TRANSACTION_REPEATABLE_READ
0: jdbc:hive2://master01:10000> show tables;
+-----+-----+-----+-----+
| database | tableName | isTemporary |
+-----+-----+-----+-----+
| default  | src       | false       |
+-----+-----+-----+-----+
1 row selected (0.726 seconds)
0: jdbc:hive2://master01:10000>
```

Sparksql 服务跟 hive 整合详细步骤：

1. 将 `hive-site.xml` 放入 spark 安装目录的 `conf` 中
2. 将 `core-site.xml` 放入 spark 安装目录的 `conf` 中

3. 启动 sparksql 的 thrift 服务进程

```
sbin/start-thriftserver.sh
```

4. 然后就可以用支持 jdbc:hive2 的客户端（比如 beeline）去连接了

```
(1) bin/beeline -u jdbc:hive2://localhost:10000 -n root
```

自己写代码客户端去连接 sparksql 的 thriftserver

